

分类号: _____

密级: _____

UDC: _____

编号: _____

工学硕士学位论文

高性能 WEB 防火墙性能评价及优化技术研究

硕士研究生 : 徐 志

指导教师 : 杨 武 教授

学科、专业 : 计算机科学与技术

论文主审人 : 王 巍 副教授

哈尔滨工程大学

2015 年 3 月

分类号: _____

密级: _____

UDC : _____

编号: _____

工学硕士学位论文

高性能 WEB 防火墙性能评价及优化技术研究

硕士研究生 : 徐 志

指导教师 : 杨 武 教授

学位级别 : 工学硕士

学科、专业 : 计算机科学与技术

所在单位 : 计算机科学与技术学院

论文提交日期 : 2014 年 12 月

论文答辩日期 : 2015 年 3 月

学位授予单位 : 哈尔滨工程大学

Classified Index:
U.D.C:

A Dissertation for the Degree of M.Eng

The Research of High-performance WEB Firewall
Performance Evaluation and Optimization
Technology

Candidate: Xu Zhi

Supervisor: Prof. Yang Wu

Academic Degree Applied for: Master of Engineering

Specialty: Computer Science and Technology

Date of Submission: December, 2014

Date of Oral Examination: March, 2015

University: Harbin Engineering University

摘 要

技术的进步，也带动着网络安全问题的加深。WEB 防火墙作为一个安全工具，对网络通信有着重要的影响。WEB 防火墙是一个对 HTTP 数据进行深入分析的防护系统。WEB 防火墙系统作为网络安全的保护者同时，也容易成为网络通信的性能瓶颈。因此，WEB 防火墙的测试及性能评价，以及相关的优化技术就显得尤为重要。本文从 WEB 防火墙的性能评价方法，以及具体的性能优化技术两个方面进行研究。

第一方面是性能评价。在性能评价的理论研究方面，通常以数学上的排队论理论对其进行研究。目前对防火墙系统的排队论研究，只是将防火墙系统看作一个整体，或者只是简单对系统的一个层次进行研究。实际上，WEB 防火墙是一个三层结构，由 IP 层、TCP 层以及 HTTP 层三层组成。因此，本文提出一种分层 WEB 防火墙系统排队论模型。该模型将 WEB 防火墙系统进行了三层细分，对它们分别运用排队论进行研究。三层之间通过流量进行关联。基于此模型，主要研究了在防火墙硬件资源限制的情况下，WEB 防火墙系统的吞吐量以及三层服务台的数量如何分配问题，最后能够得到资源最优的分配和吞吐量的最大值。

第二方面是性能优化。针对 WEB 防火墙系统的性能优化，本文也从两个方面进行深入分析。第一点是关于动态内存管理。对于传统的内存管理存在很多的方法，如位图法、空闲表法等。目前，对空闲表法中内存池技术进行的研究是去除了链式指针处理，进行了优化。本文在其基础之上，深入研究了动态内存管理技术，结合栈比队列的局部性优化的特点，对其算法进行了改进，实验证明改进后的性能得到了较好的提升。另外研究的是关于 WEB 防火墙系统中的哈希冲突解决方法的改进。传统的哈希冲突采用的是链表的形式，以头结点的形式插入解决冲突，没有很好的利用网络流的特点。目前的研究是在其基础之上，结合网络流的特性，进行优化改进，本文在改进的基础之上，利用空间换时间，增设缓存指针的方法进行了进一步的优化改进，实验证明进一步改进后的算法在性能上取得了良好的优化。

关键词：WEB防火墙；排队论模型；网络流局部性；内存池；哈希冲突

Abstract

With the development of the technology, network security problem also becomes more and more important. WEB firewall as a security tool, plays an important role in the network communication. WEB firewall is a safeguard system that analyses the http data deeply. WEB firewall system is a protector of the network security. Meanwhile, it also is easy to become the network communication performance bottlenecks. As a result, the testing of WEB firewall and performance evaluation, and related optimization technology is particularly important. This paper will study the WEB firewall system from the two aspects, the method of the performance evaluation and the technology of the performance optimization.

The first point is the performance evaluation of WEB firewall system. About the theory research of performance evaluation, the queuing theory in mathematical theory is usually used. At present, in the research about queuing theory in the WEB firewall system, the WEB firewall system is looked as a whole or the only one layer is simply researched. In fact, WEB firewall is a three layers structure. It consists of IP layer, TCP layer and HTTP layer. Therefore, this paper proposes a layered queuing model for WEB firewall system. In the model, WEB firewall system is subdivided into three layers. The three layers are studied by queuing theory respectively. They are connected by the traffic. Based on this model, the throughput of WEB firewall system and the number of the three layers of reception how to set are studied in the situation that the hardware resources of the firewall limit. At last, the optimal allocation of the resource and the maximum throughput of the system can be got.

The second point is about the performance optimization of WEB firewall system. For WEB firewall system performance optimization, this paper also analyzes from two aspects. The first is about dynamic memory management. For traditional memory management, there are many methods, such as bitmaps, leisure table, etc. The present memory pool technology in the leisure table was improved by removing the chain point-handling. However, this paper optimizes the algorithm by being based on the result, and further studying the technology of dynamic memory management, and combining with the locality of the stack better than the queue. It is proved by experiments. Then it is the improvement the hash conflict resolution method in the WEB firewall system. Traditional hash conflict resolution method is the form of a list by inserting the node to the head of the list. It isn't a good method, and doesn't take the advantage of the characteristics of the network traffic. At present, the optimized method is combined with the locality of the network flow on the basis. This paper is based on the

improved method, and uses the idea space for time to improve the improved method by adding buffer pointers. It is proved that the further improved algorithm achieves good optimization on performance by experiments.

Key words: WEB firewall; queuing model; network traffic locality; memory pool; hash collision

目 录

第 1 章 绪论	1
1.1 课题研究背景	1
1.2 课题研究的目的是意义	2
1.3 国内外研究现状	3
1.4 论文研究内容与组织结构	3
第 2 章 相关理论研究	6
2.1 引言	6
2.2 WEB 防火墙性能相关指标	6
2.3 排队论理论	7
2.4 动态内存管理技术	11
2.5 哈希冲突解决技术	14
2.6 时间/空间局部性原理	16
2.6.1 系统局部性	16
2.6.2 网络流量的局部性	17
2.7 本章小结	18
第 3 章 面向 WEB 防火墙排队论模型研究	19
3.1 引言	19
3.2 /M/M/c/K 混合方式排队系统	19
3.3 WEB 防火墙系统分层排队论模型创建	19
3.4 WEB 防火墙系统吞吐量分析	22
3.5 模型验证	23
3.6 本章小结	29
第 4 章 一种面向 WEB 防火墙性能优化方案	31
4.1 引言	31
4.2 WEB 防火墙优化方案设计	31
4.2.1 内存池优化技术	31
4.2.2 哈希表查找优化技术	34
4.3 实验及结果分析	36
4.3.1 内存池优化实验	36
4.3.2 哈希表查找优化实验	39
4.4 本章小结	44

结 论.....	45
参考文献.....	46
致 谢.....	50

第1章 绪论

1.1 课题研究背景

随着 TCP/IP 协议在网络中的使用以及现代网络和通信技术的发展, 互联网技术盛行, 网络与人们的生活越来越相关。早期的用户和主机之间互相信任, 而现今互联网上的每一个人都有可能遭遇到安全风险, 很容易遭受到恶意软件和黑客们等的攻击。而且网络垃圾信息会在不经意间侵略人们的计算机, 影响人们的正常上网。

2013 年网络谣言、诈骗和信息泄露等事件的频繁发生, 影响了互联网健康正常的发展。中国网民信息安全相关研究报告, 显示在 2013 年下半年, 上网网民中 74.1% 遇到过网络安全问题, 影响人数近 4.38 亿^[1]。其中, 有成龙慈善基金会网站、.cn 域名、BTC China 等遭受攻击, 这些攻击的形式变化多样; 中国人寿公司、如家等商业酒店、圆通快递快件公司的信息泄露等, 信息泄露似乎日常可见; 关于“7.23”动车事件、陕西天然气涨价、延安列车轨道被暴雨冲毁等谣言, 在网络中日益繁盛; 因为“套餐窃贼”, 用户信息被盗, 像“支付鬼手”影响着网络支付的安全等, 网络安全形势不容乐观^[2]。

在央广网上, 北京时间 2014 年 11 月 28 日发布一条消息^[3]。其称根据国家互联网应急处理中心统计, 当前的恶意具有破坏性的程序的数量日趋增长, 就在今年上半年的时候, 发现的恶意的程序多于 36 万, 相比 2013 年同期增加了 17.7%。在这些恶意的程序当中, 恶意扣除费用这一类, 所占比例超过了一般达 55%, 位居榜首, 次一位的是远程控制类占 16%, 在接着的是占 8% 的信息窃取类。在去年 8 月份的时候, 我国 CN 顶级域名也遭受到黑客的攻击, 致使 23 个省份使用的 CN 国家域名的网站出现了长达两小时瘫痪的现象, 此外中国政府网站、中国银行网站、中国电信等网站都受到了相对的攻击。今年 6 月份的时候, 人民网再次受到攻击, 攻击者竟然是一个不到 18 岁的男孩, 据说是为了锻炼自己的科技水平。今年 8 月份的时候, 随着手机的普遍使用, 手机上的漏洞也爆发了, 10 月份, 中联办网站因为受到攻击瘫痪。还有就是钓鱼网站的数量也在逐渐增加, 已经监测到的有 7964 个, 其中还有大量未监测到的情况, 这些钓鱼网站主要是针对我国境内的钓鱼网站, 这些被植入后门的网站之中, 有 84.8% 被不明的境外 IP 所控制。

网络安全越来越凸显重要。正因为它的重要性, 所以它也越来越被关注。早在 1994 年的 IAB (Internet 体系结构委员会) 的一次研讨会上, 安全与扩充问题就被提出当做最重要的两个问题进行了讨论^[4]。

早期的防火墙和计算机之间并没有什么关系, 它是一种保护房子等建筑的工具。随着现代计算机技术和网络技术的发展, 借用了这个概念, 使用防火墙来防御有害的信息传播以及来自外界的入侵, 保护数据信息的安全。

最早防火墙的概念是产生于 1986 年，随后其便在计算机网络中开始兴起和发展。传统防火墙一般情况下分成四种分别为包过滤型、状态动态监测型、应用程序型和个人的防火墙等。随着处理安全相关问题的深入，处理的层次更高，后来又产生了 WEB 防火墙^[4]。

防火墙系统按照其实现处理的 OSI 模型（Open System Interconnection，开放系统互联参考模型）的层次级别，又可以被分成三种为数据包级过滤型、应用层级过滤型，以及两者功能兼具的复合型^[5,6]。

WEB 防火墙系统相对于传统的防火墙系统的区别在于，传统的防火墙一般工作在 TCP 层或者以下，而 WEB 防火墙则工作在更高的应用层。传统防火墙只对 TCP 头部、IP 头部进行解析，并不深入到应用层数据。WEB 防火墙需要对 HTTP 请求和响应数据进行解析、重组处理，然后对 HTTP 内容进行设定的特定匹配规则和攻击特征进行数据匹配，实现对 WEB 攻击行为的监测和做出相对应的处理，如拦截、报警或者记录日志等方法。因为传统的防火墙处理的深度比较浅，因此处理的速度快，而 WEB 防火墙因为其处理层次更高的原因，其相对的处理速度、性能等会有所下降，因而容易变成网络通信的性能瓶颈^[7]。

1.2 课题研究的目的和意义

新技术的不断升级，攻击的变化越来越多样，传统的、简单的、低层次的数据包过滤级的防火墙技术，已经不能够满足要求，难以对数据和信息安全进行保护，所以需要数据的处理更加深入，深入到更高层，应用层级别。但是，因为对应用层的深入数据解析、处理，它的处理时延必然增加，处理的效率则会相应的降低。此时的 WEB 防火墙系统的性能如何，能否满足网络环境的需求，如何去测试和评价一个 WEB 防火墙系统便变得更加重要，需要保证 WEB 防火墙的应用不会成为一个性能瓶颈。

本课题通过研究 WEB 防火墙性能评价技术，旨在确定防火墙的性能，评价当前的防火墙系统是否满足需求，以及确定防火墙系统性能的影响因素。

当然，当在对 WEB 防火墙系统进行性能评价的时候，发现 WEB 防火墙系统性能成为了网络性能的瓶颈，这时候又当如何去处理 WEB 防火墙系统。自然离不开关于 WEB 防火墙系统的性能优化，而不是舍弃它而不使用。这就需要对其进行有关性能优化的研究。

如何结合创建的评价技术系统来评价一个防火墙系统，以及结合性能优化技术对系统进行性能优化，这也是研究系统性能评价技术和性能优化技术的意义所在。评价技术通过指导 WEB 防火墙系统中存在的影响系统性能的关键因素，然后通过优化这些影响系统的关键因素，来提高系统的处理性能，达到优化全局系统的目的。

1.3 国内外研究现状

在 WEB 防火墙相关的领域,对系统性能评价研究,还比较简单。针对网络相关的性能评价,目前较常使用的方法有三种^[8]。这三种方法如果引用到 WEB 防火墙系统性能评价,将会表现出如下的三种形式:第一种形式是,对 WEB 防火墙系统本身进行测量,然后收集采集各种参数指标,对收集的参数指标进行分析,然后对 WEB 防火墙系统做出一个整体的性能评价;第二种形式为,为 WEB 防火墙系统抽象出一个数学模型,通过求解公式的方法,来说明一个 WEB 防火墙系统的优劣,进行评价。第三种便是,通过编写计算机模拟程序对 WEB 防火墙系统进行模拟测试,然后得到模拟测试结果,分析这个结果,然后去评价一个 WEB 防火墙系统的性能。

目前针对防火墙的性能评价技术研究,主要就是上面述说的第一种和第二种方法。比如哈工大的田原,主要是从第一个角度出发,分析影响防火墙性能相关因素,结合性能评价的相关标准、测试策略等,提出了两种测试系统的方案^[5]。其主要目的在于测试和获取防火墙的相关指标。有部分研究者以排队论的相关知识基础为背景,将其引入到防火墙系统中,对防火墙系统进行研究^[9,10,11,12]。排队论模型将整个防火墙系统看做一个服务台或者说是多个服务台,来的数据包看做是消费者,或者说是顾客。消费者和服务台之间便形成了一个排队系统,其又被称之为随机服务系统^[13]。基于排队论推导出有关的公式,比如说时延公式、吞吐量公式、丢包率公式以及并发连接数公式等,然后利用它们分析系统性能。

其实在防火墙系统中,时延一直备受关心。Hoffman D 等人指出在 linux 系统中,防火墙的时延和帧长以及规则的大小成一定的正相关关系^[14],Yuan L H 等通过建立模型对防火墙的时延进行分析^[15]。Lyu M R 等指出防火墙时延长不一定安全系数就高^[16]。前面的那些人研究只是针对传统的 IP 层的防火墙,停留在包过滤的层次。如何缩短防火墙的处理时延,尤其是 WEB 防火墙的处理时延,提高 WEB 防火墙的处理效率,一直是研究的方向,需要进一步深入的研究。因为相对于传统的防火墙,WEB 防火墙工作的层次更高,处理的流程更加的复杂,导致其时延也相对应的增长,自然,为了使 WEB 防火墙不成为性能瓶颈,就要对其优化。一般进行配置规则优化^[17,18,19,20],此外,优化的方向还有两个一个是关于内存的管理^[21,22],一个就是管理流表和会话表的管理^[23,24],一切的目的都在于提高系统的性能。

1.4 论文研究内容与组织结构

通过阅读和参考大量的性能评价技术以及相关的排队论的知识,本文将主要结合排队论的理论知识来分析 WEB 防火墙系统,因为 WEB 防火墙系统是一个分层的结构,所以对于系统的分析也进行分层处理,从而建立分层排队论模型。通过阅读和参考大量有关防火墙技术和 HTTP 协议处理技术的文献资料,本文将以传统的内存池技术和传统

的哈希表链式冲突解决技术作为研究的基础，结和时间、空间局部性原理，包含网络数据流的局部性，外加存储器的层次结构等特性，对现有的方法进行了优化，来提高程序的处理效率。本文所做的工作为以下三个方面：

1) 研究 WEB 防火墙及其性能评价技术，目前从理论上对防火墙系统的研究主要就是引入排队论理论，但是目前的排队论理论仅仅将防火墙系统视为一个整体，或者仅仅对其中的一个层次进行简单的分析^[9,10,11,12]，并没有深入分析 WEB 防火墙系统的层次架构。WEB 防火墙系统是一个三层的架构结构，所以为了更好的分析 WEB 防火墙，本文在前人研究的基础之上，提出了一个 WEB 防火墙的分层排队论模型。基于此模型，深入研究了有关 WEB 防火墙在受到计算机资源限制的条件下，如何最优分配各层的服务台数，以使 WEB 防火墙系统处于最优的状态，从而使它的吞吐量达到最大。

2) 研究现有的动态内存管理技术，以及基于现有的内存池技术的改进，针对链式结构管理内存池的缺点，有研究者提出以数组下标索引的方式^[25]，管理内存池，但是在对索引管理的时候，队列并不是好的方法，它在系统的局部性上表现出较差的性能，相反栈结构的后进先出的特点，则表现较好的局部性，因此，在该算法的基础上，修改了索引的存储结构，使得处理的效率和性能得到提高。

3) 研究现有的哈希冲突解决技术，以及网络流的特性。有研究者做过统计表明，网络流的到达符合一定的规律——泊松分布^[26]，在一定程度上表现出局部性的特性^[27]，因此，有研究者，在此研究的基础上，对传统的链式哈希冲突解决技术，在网络流量处理的环境下进行了改进，提出了移动当前匹配结点到冲突链表表头的技术^[23,28]，但是这并不能很好的使局部性这个特性发挥的更好，因此，在其研究的基础上，再次综合系统中存在局部性的特点，使用空间换时间的方法，引进了几个缓存指针，指向最近匹配到的冲突结点，减少处理的过程，从而提高程序的处理效率。在改进的方法中，进一步使用了栈的后进先出的良好的局部性的特性。

根据所要论述的内容和系统的实现，论文的组织结构如下图 1.1 所示。

第 1 章说明了本文的研究背景，目的和意义，还有当前的研究情况，本文的内容及结构等。

第 2 章研究了关于 WEB 防火墙的相关理论，分析了其性能评价指标、排队论理论、动态内存管理技术、哈希冲突解决技术以及时间/空间的局部性原理。

第 3 章研究 WEB 防火墙性能评价，提出了关于它的分层排队论模型。并在此模型的基础之上，对 WEB 防火墙系统吞吐量和各层服务台的最优分配进行了研究，最后得出最大吞吐量的值和各层服务台的最优分配。

第 4 章对 WEB 防火墙系统性能优化进行研究。主要从两个方面着手进行。第一个方面是关于内存管理，第二个方面是关于哈希表的哈希冲突链的处理。这一章主要是将局部性原理技术，引入到 WEB 防火墙性能优化中，来提高其处理性能。

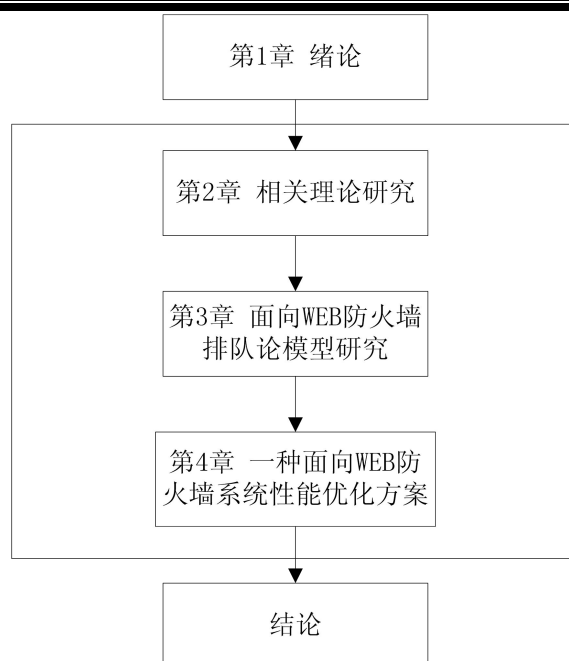


图 1.1 论文的组织结构

最后，在本文结束处，对本文所做的研究工作进行了一个总结，分析了本文在性能评价的分层排队论模型以及性能优化方法上所做的工作。

第 2 章 相关理论研究

2.1 引言

WEB 防火墙就是工作在应用层的防火墙, 相比较传统的防火墙而言工作的层次更高, 能够处理的内容更加的丰富, 能够更加深入的处理应用层的数据。WEB 防火墙的实现, 是基于 TCP/IP 协议的, 从底层获取网络中的数据, 然后进行 IP 层处理, TCP 层处理, 然后提交应用层, 进行 HTTP 协议数据解析, 然后进行规则匹配, 深入分析应用层数据的安全性。

实现一个系统是基础, 如何去评价一个系统的优劣, 处理性能的高低、好坏, 是需要进一步的研究, 因为只有在评价的基础上, 才能更好的去改进优化一个系统。让系统的更加健壮和完备, 更好的工作。下面将会从性能测试评价指标、排队论理论、动态内存管理技术、哈希冲突解决技术、时间空间局部性原理等这些方面进行相关理论研究。

2.2 WEB 防火墙性能相关指标

WEB 防火墙性能测试的过程, 就是参照一定的防火墙相关的指标标准, 从各个方面其性能进行测试评估。

IETF (Internet Engineering Task Force, 互联网工程任务组) 进行相关标准化工作, 并且写成了相关方面的 RFC 文档, 来指导防火墙测试工作的进行。在讨论防火墙测试之前, 需要先了解一下网络的指标, 因为防火墙的测试是基于网络的。与防火墙测试相关的很多的概念离不开网络的 RFC 文档。RFC1242 文档说明了一些基本术语, 用来描述测试性能相关的标准和有关测试结果的标准, 其中提到的一些指标有吞吐量、时延、丢包率等^[29]。

1) 吞吐量

在 RFC1242 文档中, 对于吞吐量的定义是: 正确转发所有数据的最大速率。在防火墙系统中, 对于吞吐量的定义文档 RFC2647 这样写到^[30]: 单位时间内, 转发到防火墙的正确的目的接口的位数, 其中减去丢失和重传相关的位数。吞吐量是防火墙系统主要的测量指标之一, 它能够描述有多少数据量或者数据包通过了防火墙。它的单位一般是包/秒 (也即 pps), 或者位/秒、字节/秒, 也有用帧/秒等。

对于 WEB 防火墙, 因其特点, 应用层的吞吐量也需要被考虑。所谓应用层的吞吐量, 就是只对有效的 HTTP 数据进行统计。总得来说, 吞吐量很大程度上反映着一个防火墙系统的处理转发能力。在性能测试的时候, 是不可或缺的。

2) 延迟

在 RFC1242 文档中, 设备对数据处理存在两种情况: 存储转发和按位转发。第一种

设备延迟的定义是输出第一个比特位的时间与进入最后一个比特位的时间之差。第二种设备延迟的定义是输出第一个比特位的时间与进入第一个比特位的时间之差。网络中时延的产生有很多原因，如发送、传播、处理以及排队等导致。对于 WEB 防火墙系统，更多的关心的时延主要是处理时延和排队时延。在网络应用中，有很多应用对时延的要求很高，比如视频传输、音频传输等，对时间非常敏感，对于在网络通信线路上加入了防火墙，尤其是 WEB 应用层防火墙，它是不是能够很好地满足网络的需求，这一点测试非常的重要。当然了，时延越小，自然 WEB 防火墙的系统的性能就会越好，更好的满足需求。但是更多的时候，因为处理的原因，其时延不能避免。设计优化的时候，在如何减少处理时延的时候，考虑更多的应该是减少过程的调用，以及算法的优化，或者利用存储器的层次结构等。

3) 丢包率

在 RFC1242 文档中，介绍了丢包率。其定义是在一种稳定的状态下，因为缺少资源，本该转发的数据包，却没有被转发，而产生了丢包的情况，丢失的数据包占有所有数据包的比例，称之为丢包率。丢包率和吞吐量之间存在关联性。简单的说，就是理论上，网络流量没有达到吞吐量的时候，系统的丢包率应该为零。只有到达流量超过吞吐量时，系统才会开始产生丢包的问题。

2.3 排队论理论

在现实生活中排队的现象到处可见，出现这样的原因主要是由于资源不足等引起的，如服务机构管理低效、服务员的效率差、又或者顾客不遵守纪律等原因也可能出现不该存在的排队的现象。

有学者将现实生活中的排队现象，进行抽象，引入到了研究当中，产生了一门数学理论便是排队论。这些排队的现象都可以总结概括为顾客和服务台之间的某种服务关系，抽象成如下图 2.1 所示的排队过程^[31]。

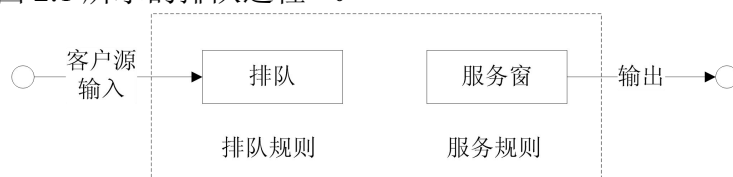


图 2.1 排队的模型框图

不同形式的排队系统都具有随机性^[20]。在顾客到来的时间差值和其所需要的处理时间之中，至少需要一个表现出随机性，否则，表现出的排队问题就相对容易了。排队理论所分析的就是排队系统参数指标的概率特性，主要包括三个部分^[32]。

第一，形态问题。所谓形态问题，就是各种概率问题。它涉及排队的队长、在队列中等待和及系统逗留的时间，以及忙碌的时候等概率表现情况。这种情况有两种形式，一种是瞬间表现出的形态，另一种是统计稳定状态下的形态。排队理论研究的核心问题

就是这种表现的概率形态,它是统计推断以及最优化问题求解的前提。在可实用性方面,统计稳定状态下的指标的概率分布比瞬间的概率性质表现的更加重要。

第二,数据处理。为了更好的了解和正确认识一个正在使用的排队系统的表现出的规律性,通过一次简单的观测得到的数据并不能说明任何的问题,需要反复多次的观察测量,采集数据,然后对这些数据运用数理统计学分析处理,然后推测得到该排队系统表现出怎么样的概率形式,以便决定应该使用怎样的理论模型去分析和处理当前系统相关问题。对排队系统中数据进行统计分析是一件基础的工作,因为这样才能更好的找到相关的理论成果对其进行分析。

第三,就是最优化问题。关于最优化问题包含两点,其一为如何最优化设计一个排队系统,其二,如何对已经存在的排队系统在运行中保持最优控制。前者是在系统的设计之前,分析存在的数据,对将要出现的情况进行预估,是设计的系统表现出最优的情况。后者是对正在使用的系统找到一种最优的处理方法。对于一个排队系统来说,系统设计以及系统运行时的管理,需要同时考虑到顾客和服务两者之间的利益关系,以便在合理处理系统中一些指标的时候,使得排队系统达到一种最优的、最理想的状态。在大多数系统中,一般输入是由客观存在的条件决定的,则处理优化问题的关键之处就在于服务这一边了,如服务台数的多少、服务的效率的高低以及服务的方式的选择等等。在做优化处理的时候,优化的指标参数,可以使从时间、费用、空间资源等角度进行。

排队系统一般由三个部分组成,即输入过程、排队规则以及服务台。

1) 输入过程是说明顾客出处以及来到的一种规律。

- a. 顾客总数可有限,也可无限;
- b. 顾客来到的可以是逐个到达,也可以是成批到达。
- c. 顾客来到的时间上的差异可确定,也可随机;
- d. 顾客来到可独立,也可相关,输入的过程表现出不同的特性其次、平稳又或者为马尔科夫等。

2) 排队的规则,是服务台对可排队的顾客的顺序和形式的一种要求,其表现出三种形式。

- a. 损失的方式:当顾客来到服务地点的时候,发现存在的服务台,已经被占满,没有空闲的位置,此时,它的表现是立刻离去。在现实系统中有很多这样的情况,像住宿的旅店已经被注满的情况等。
- b. 等待的方式:当顾客到达服务地点的时候,发现所有的服务台,都被占用,没有空闲的位置,但是在服务的地点设有等待的座位,以使来到的顾客能够排队等待,这样它并不是立即离去,而是依照顺序进行排队等待。这种顺序呈现出多种情况,分为:先来先处理,也就是接受处理的顺序是依据相应到来的先后顺序进行;后来先处理,和前者相反,表现出后到的顾客先进行处理;随机

处理,服务台在排队的顾客中随机挑选其中的一个进行处理;有优先权值的先处理,参与排队的顾客有些表现的特别,需要给予最高的权值,让他们先被处理,对于有优先权值的服务方式表现出两种形式,分为强拆形式和非强拆形式。所谓强拆形式就是当具有高优先权的顾客到来时,正在接受处理的顾客的处理立即停止,让它先被处理。非强拆形式优先权就是当具有高优先权的顾客到来时,并不是立即被处理,只有正在接受处理的顾客结束时,它才开始被处理。

- c. 等待和损失的混合方式:分为三种形式,一等待队长有限的混合方式,这种方式表现是服务系统为顾客提供进行排队等待的座位,但是提供的座位的数量有限,因此当排队的座位也被占满的时候,顾客只好离去。二有等待时间有限制的方式,表现出顾客看到排队的队列太长,不愿排队的现象。三就是逗留时间有限制的方式,相对于第二个多了处理时间。

3) 服务台,对到来的顾客进行处理

- a. 在系统中,服务台可能是一个或者多个,还有可能是零个的状态。
- b. 对于多服务台这种情况,顾客可以有多种排队的形式,如并行的排成多个队列的形式,又或者只是简单的排成一列,或者两种形式同时存在的混合队列。
- c. 一个服务台表现出的服务形式是可以为一个顾客提供服务,也可以为成批顾客进行服务。
- d. 多个窗口之间的处理时间表现可能是明确的,也可能是随机的,但是一般情况下,假定是明确的状态。

对于排队系统,一般使用肯达尔记号进行描述^[33,34],即使用三到五个英文字母来表示,字母之间使用斜线相隔。如/A/B/C/D/E,其中A、B、C、D分别表示是输入的情况、服务时间的情况、服务台的数量、系统队列的长度,E可能存在,也可能不存在,存在时一般表示的是顾客流中顾客的类型数。

在排队论系统中,对于其描述含有四个指标。

其一为队列及等待队列的队长。队列的长度是指等待服务的顾客数量,而等待队列的长度是指系统中总的顾客数量,它包含系统等待服务的以及正在接受处理的顾客的总的数量。这两个参数都是随机值,是排队系统中的重要参数。

其二为等待时间和逗留时间。所谓等待时间,就是顾客在队列中消耗的时间,而逗留时间是指在系统中消耗的时间,即进出系统的时间的差值,它是等待和服务时间的总和。若顾客的到来和接收的服务在表现形式上是不相关的话,等待与逗留的时间也是彼此不相关的。两者是顾客关心的指标参数,在实际使用的时候,统计的是稳定的状态下的时间分布以及期望值。

其三是忙碌期和空闲期。所谓忙碌期，是指系统从空闲的状态结束，到再一次进入此状态的时间之差，这个反映的是服务台的工作强度。所谓空闲期是指两次忙碌状态之间的状态，系统持续一种空闲状态的时间值。一般到达稳定情况下，两者是轮流出现的。

其四是输出过程，就是顾客在被系统处理结束之后，离开的一种方式。对其描述主要的参量就是两个顾客离开的时间之差和一定的时间之内离开的顾客数量。这些值，反映着系统的工作效率。

在排队理论研究中，关于概率的研究表现的非常的重要。以下说明一下重要的概率分布。

1) 负指数分布

定义 1: 对于一个变量 x ，如果满足，

- a. 连续且随机，
- b. 分布密度函数为

$$f(t) = \begin{cases} \lambda e^{-\lambda t}, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad (2.3.1)$$

其中 λ 是大于 0 的常数，则称变量 x 是服从 λ 的负指数分布。其概率分布函数是

$$F(t) = \begin{cases} 1 - e^{-\lambda t}, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad (2.3.2)$$

它的 k 阶原点矩 $E[X^k] = \frac{k!}{\lambda^k} (k=1,2,\dots)$ ，方差 $D[X] = \frac{1}{\lambda^2}$ 。

定义 2: 假设 $E[X] \neq 0$ ，随机变量的 x 的变异系数为

$$\phi = \frac{\sqrt{D[X]}}{E[X]} \quad (2.3.4)$$

变异系数为 1 是一个随机变量 x 服从负指数分布的必要条件。

2) 泊松分布

定义 3: 对于一个变量 x ，如果满足，

- a. 离散且随机，
- b. 概率分布密度函数为

$$p_i = P\{x=i\} = \frac{\lambda^i}{i!} e^{-\lambda}, i=0,1,2,\dots \quad (2.3.5)$$

其中 λ 为大于 0 的常数，则称变量 x 是服从 λ 的泊松分布。其期望值为 $E[X] = \lambda$ ，方差为 $D[X] = \lambda$ 。

3) 泊松过程

它又被称为泊松流或者最简单流。对于单个类型顾客， $N(t)$ 表示在时间 $(0, t]$ 内到来的数量， $\{N(t), t \geq 0\}$ 是泊松过程需要满足下面四个条件：

- a. $\{N(t), t \geq 0\}$ 是连续时间参数的随机过程

- b. $N(0) = 0$
- c. $\{N(t), t \geq 0\}$ 增量是独立的, 即对任意取值的 n 个时刻: $0 < t_1 < t_2 < \dots < t_n$, 变量 $N(t_1) - N(t_0), N(t_2) - N(t_1), \dots, N(t_n) - N(t_{n-1})$ 都具有独立性。
- d. $\{N(t), t \geq 0\}$ 增量是平稳的, 且对于任意的 $t \geq 0$ 和 $s \geq 0$, 有

$$P\{N(t+s) - N(t) = k\} = \frac{(\lambda s)^k}{k!} e^{-\lambda s}, k = 0, 1, 2, \dots \quad (2.3.6)$$

其中 λ 为非负常数。

2.4 动态内存管理技术

在 C 程序设计中, 常常会出现关于内存分配问题。可以通过程序的编译和链接为结构体、数组等数据结构分配内存, 但是在程序中, 通常对内存动态的操作是 `malloc` 和 `free` 函数, 应用程序通过调用它们来申请和释放内存空间。 `malloc` 和 `free` 的实现是调用了系统函数, 会与内核产生交互, 这样便产生了上下文切换。如果频繁调用, 则会产生大量的上下文切换, 占用大量的 CPU 时间, 致使应用程序的效率低下。不仅如此, 频繁使用 `malloc` 和 `free` 函数, 最终也会导致内存碎片的产生, 致使内存的使用率降低。

在内存中, 会存在内部和外部碎片两种情况。外部碎片就是系统中空闲块和占用块之间相互交错, 虽然总的空闲块大小能够满足新提出申请内存的需求, 但是每个空闲块的大小又太小, 不足以满足提出的内存申请需求, 这样便产生了外部碎片, 致使内存空间无法使用。外部碎片的产生使内存空间申请失败, 利用率降低。对于内部碎片的产生, 源于实际分配的空间比实际需要的要大。造成这种情况的原因有很多。第一种原因是内存分配的时候遵循一定的准则, 如边界对齐的缘故 (即所有的内存起始地址, 必须是 4、8、16 或 32 的整数倍等), 这是因为内存管理的时候为了提高访问速度。第二种原因是内存管理的时候强加的情况, 比如内存申请的空间不能小于某一个值, 如果小了自动取当前设置的最小值。第三种原因就是有可能存在对于每一个内存块使用情况的说明信息 (如空闲块的位置、大小以及所有权等)。上面只是列出了比较明显的三种情况, 可能还存在其他的原因。通过上述可见, 碎片使得内存的使用率降低。

还有对于 `malloc` 调用, 如果申请的空间没有被释放, 那便会产生内存泄露, 内存泄露将会导致内存被耗尽, 没有内存可以使用, 程序执行效率越来越低下, 最终可能程序崩溃。如何改进和优化内存处理便成了一个思考的关键问题。

对于内存的管理存在很多的方法。如: 位图法、可利用空间表法等^[35]。

位图法, 简单的说就是将内存使用情况, 映射到位图上, 位图中的一位和可以分配的内存相互对应。在位图中, 0 标识未使用, 1 标识使用 (或者 0 标识使用, 1 标识未使用)。一块内存空间及其位图如下图2.2所示。其中深色和白色分别表示被使和未被使用的内存块。

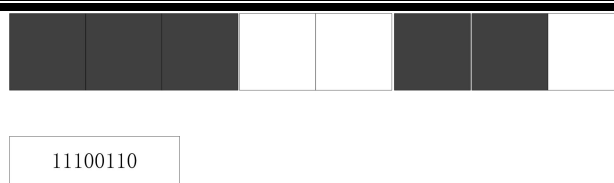


图2.2 位图和内存对应关系

在位图法中内存块划分的大小是一个重要的因素，影响着位图的大小。划分的内存块大小越小，则划分的数目就越多，对应着位图的位数就越大。例如，按 4 字节 32 位划分一个内存块，一个 32 位的内存块，其对应位图中也只需要一位，32N 位的内存空间，其对应的位图位数需要 N 位。可以计算出来，位图占用了 $1/32$ 的空间大小。如果按 2 字节 16 位划分，32N 位的内存空间，其对应的位图位数需要 2N 位。通过计算位图占用了 $1/16$ 。对比，可以看到，内存块大小划分的越大的话，位图的空间就会越小，位图占用空间比也就越小。这一点说明因为存储内存相关信息耗费的内存小，但是内存块划分的大，也存在一个缺点，就是内存分配的时候，如果实际需要的内存比内存块大小要小，那么就产生了内存碎片。所以块的大小的划分，也需要进行考量。

位图法利用一小块内存提供了一种非常简单的描述着内存空间的使用情况。位图法也有它自身的缺点，就是在内存分配的时候，需要对位图进行搜索。比如，现在需要分配 M 个内存块，那么就需要在位图中找到连续 M 个标识没有被使用的内存块。因为可能存在连续的未使用标识并不在一个字中，所以查找操作是一项耗时操作，这是位图的缺点。

可利用空间表，也被叫做内存池技术。可利用空间表根据系统运行的相关情况，表现出三种不同的结构形式^[36]。

第一种情况，就是用户在程序处理之前，初始化的时候，预先申请一块大的内存空间，将这一块内存空间分成若干份相等的内存块，这样每次内存申请的时候，并不是再向操作系统申请内存，而是直接从初始时的内存空间中获取，释放的时候，也不是将需要释放的内存还给操作系统，而是将其放入到初始时申请的内存空间中。这样便能减少每次内存处理与操作系统的交互。其中，更多的表现形式是把这些相等的内存块以指针链的形式，连接成一个可用空间的链表。分配空间的时候，不需要查找，直接取出第一个结点使用即可；释放空间的时候，也无需进一步查找，直接链入链表的首部即可。可以看到，这其实是一个链表栈，减少了查找的时间。

第二种情况，系统在运行的时候，用户请求的内存空间不是单一的，而是存在若干大小变化性的内存。对于这样的情况，一般的解决的方式是，建立多个空闲链表链，同一链表链中表现的情况就像第一种情况介绍的那样，放置都是固定大小的内存块。但是和第一种情况又不同，因为和当请求的分配量的内存空闲链表中没有可以使用的内存块的时候，它会开始查找更大的内存块链表，申请一个合适的分配，取出其中一部分使用，

将剩余的空间放入到相应大小的链表中，回收时，就将其放入相应内存大小的空闲链表中。

例如，在一个系统中用户可能用到的内存空间会有 8、16 和 32 字节的内存，系统在建立内存块链表的时候，会建立三个内存结点大小为 9、17 和 33 字节的内存块链表。多分配一字节的原因，是用来存放链域（即下一块可用内存块地址）、标识域（标识该块内存块的使用情况）、结点类型域（这一类型主要用来区分链表中的内存块的大小）。

然而，这里面会存在一种比较特别的问题需要进行处理：就是在内存申请的时候，和其相匹配的空闲链结点为空，不仅如此，比其大的空闲链也为空，内存分配便不能进行，但是，有可能在内存空闲链表中存在一块连续的满足要求的内存空间，只是由于在系统的运行时，小块内存的频繁使用和回收，使得大块空间链表中的结点都被分成了小块，插入了小块空闲链表中，此时，系统如果继续正常运行，使用内存空间，就需要对内存空间进行重新整理，执行内存紧缩操作。

第三种情况，系统在运行的时候，可以给用户分配的内存块大小可以随意而变，不是固定的大小。这个时候，可利用空闲链表中内存块的值也表现一种随意性，因为存在这种随意性，分配的时候，就需要进一步进行考虑。例如，内存使用的时候，申请了一块为 `size` 的空间，如果空闲块中仅有一块（其值为 `mem`）比 `size` 的值大，那么很容易，就将它进行分配，将剩余的空间 `mem-size` 作为一个可使用的内存块结点放入链表中。但是，如果在空闲块链表中，不只有一个满足要求的内存块，而是存在多个，这样的话应该怎么分配，这就涉及到了分配策略的问题。

常见的内存分配方法存在五种。第一种便是首次符合法。所谓首次符合法，就是从空闲链表的表头开始往后查询，如果找到了符合情况的，即内存块空间大于申请的 `size` 值，便将这块内存分配给用户使用。对于空闲块链表来说，它本身之中的每个空闲块的地址并没有什么先后之别，也不按结点内存大小有序。在内存回收时，只是简单的将其链入链表的首部。

第二种就是最佳符合法。最佳符合法的关键在于找到一块大小最合适不小于请求的内存大小的一块空闲块，将其中的一部分进行分配。为了找寻这个最合适的空闲块，自然避免不了对空闲链表自首至尾的扫描。为了减少这种这种不必要的扫描操作，可以采用对链表的空闲块大小按从小到大进行排序，这样查询的时候，就不必每次都进行完全遍历链表的操作，减少了遍历的结点数，但是这样就要求，在内存释放回收，插入结点的时候，找到合适的位置进行插入。

第三种为最差符合法。这种情况和第二种方法的区别在于一个是找最合适的一块内存块进行分配，这种方法就是找到最大的一块内存块进行分配。自然，从查找时间的效率上讲，链表结构需要按照一种自大到小的顺序进行排序。这样每次分配的时候，就不

需要对链表进行遍历查找,只需要取出链表中的第一个结点便可。为了满足这样的排序,这就需要对内存释放回收时候,插入操作进行处理,找到一个最合适的位置进行插入。

上述三种算法各有自己的特点。从时间角度来说,首次符合法在查询符合内存块的时候,需要进行链表的遍历,但是再回收内存的时候,直接在链表的表头结点插入即可。最差符合法表现则相反,在内存块申请使用的时候,直接取出第一个结点使用即可,回收的时候需要对链表进行遍历找到合适的位置进行插入。所以在性能上,首次符合法和最差符合法相当。而最佳符合方法表现的最差,因为无论是分配内存和回收内存,都需要对链表进行遍历处理。从空间的角度说,最佳符合法比较适合内存大小的范围比较广的情况,因为它每次分配内存块的时候,选择的是大小最接近且不小于申请空间大小的内存块,当然由此会产生一些内存空间极小而无法被使用的小片内存块,但是也相应保存了那些存储空间比较大的内存块,以便响应后续将产生的较大内存块的申请,这样最后导致的结果,是可使用的空闲块的大小相差很大。对于最差符合方法,每次在进行内存空间分配的时候,选择的是最大的一块空闲内存空间,最后出现的结果就是可使用的内存块的大小趋于一种均匀的形态,由于存在这种特性,因此,它比较适合申请的内存空间变化范围较小的系统。对于,首次符合的方法,每次内存块分配都存在一定的随机性。

后来研究者对首次符合算法做了微小的改动,又提出了一种新的分配算法——下次符合算法。下次符合算法和首次符合算法类似,主要的区别在于,首次符合算法是从空闲链表的表头开始进行内存块搜索匹配,而下次符合算法起始搜索的位置,是上一次搜索匹配到的位置。Bays 对两个算法进行了仿真实验,实验证明下次符合分配算法相对于首次符合分配算法性能要低一点。

第五种是快速符合法,它对一些经常使用大小的空闲块进行特别处理。例如,有一个含有 M 块内存块的空闲链表,链表中的第一个结点指针指向空闲空间的大小为 1KB,第二个指向的空闲空间大小为 2KB,第三个则指向的空闲空间大小为 4KB 等,依次方式,16KB 的空闲块既可以放在 16KB 的空闲链表中,也可以放在专门存放特殊大小空闲块的特殊的链表中。快速符合方法对于找寻一个特定大小的空闲内存是非常快的,但是它也存在一个缺点,查找并且合并相邻内存块的过程十分耗时。但是,如果不进行合并的话,那么将会产生大量的小块内存,而无法被使用。

2.5 哈希冲突解决技术

在研究哈希冲突之前,先说明一下哈希表。在一种完美的情况下,可能存在一种情形就是,不需要经过比较,一次存取便能够轻松的获取要查的记录。那么就要要求在记录的地址和关键词之间存在着一种唯一的函数映射关系。依赖于这种关系,在查寻的时候,便只需要根据其函数式 f 找到这个关键词 key 相应的像 $f(key)$ 。如果存储结构中有

这样的关键词和 key 相符合,那么它一定就会在 $f(key)$ 相对应的地址之上。因此,从而知道查询的时候,查询的时候,无需进行任何的比较,便能够获取所查询的关键词记录。函数 f 被称之为哈希函数,按照上述想法创建的表结构便是哈希表。哈希表与其他的结构,如线性表、树等,最大的区别在于就是其他的数据结构数据的存储的地址位置是随机的,和关键词之间没有一种映射关系,因而在查找的时候,需要挨个关键词比较,直到查询到或查询不到所要的结果。自然,哈希表因为地址和关键词记录之间的这种关系,使得其查询的速度非常的快,时间复杂度为 $O(1)$ 。这也是其优势所在。

当然了现实中不可能这么理想和完美,在关键词和地址之间不可能完全做到一一映射。在使用哈希表的时候,难免会产生记录冲突的情况。假定哈希表的地址集为 $0 \sim (n-1)$,所谓的冲突就是指关键词经过函数计算得到的地址值 $k(0 \leq k \leq n-1)$ 上已经存在了关键词。解决冲突就是再为这个关键词寻找另一个合适的地址,或者是利用一种方法(如链表法),将其插入到当前的位置上。

解决冲突的方法存在多种,其中最常用的就是开放寻址法、再哈希法、“拉链”法以及创建一个公用缓冲区的方法^[36]。

1) 开放寻址法

$$K_i = (K(key) + x_i) \bmod n \quad i=1,2,\dots,k(k \leq n-1) \quad (2.3.7)$$

在公式中: $K(key)$ 是选择的哈希函数; n 是设置的哈希表的长度; x_i 是选择解决冲突的递增序列,一般存在三种取法。第一种为线性探测法, x_i 的取值以 1 自增的形式增加从 1 到 $n-1$ 。第二种为二次探测法, x_i 为 $\pm 1^2, \pm 2^2, \dots, \pm k^2, (k \leq n/2)$ 。第三种为伪随机探测法, x_i 的取值为一系列的伪随机的数值。

2) 再次哈希法

$$K_i = RK_i(key) \quad i=1,2,\dots,k \quad (2.3.8)$$

RK_i 表示不一样的函数,如果哈希的时候产生冲突,则使用另一个函数计算,直至不存在冲突为止。

3) “拉链”法^[37]

这是本文将要用到并且着重介绍的一种解决冲突的方法。它解决冲突的办法很简单,就是将所有的冲突结点,即经过哈希函数计算得到同一地址的结点,全部链到同一个链表上。至于新的结点,将其链到链表头部或尾部,或链到链表的中间,以便保持关键词的有序性。

4) 创建一个公用缓冲区

假设哈希表的基本表为 $HASHTABLE[0 \cdots n-1]$,经过相应函数计算得到的结果是 $[0, n-1]$ 。基表中每个值都会存放一个或多个记录,另外增设溢出缓冲表为向量 $OverflowTable[0 \cdots v]$ 。当新来的记录经过哈希计算以后发生冲突,并且每个基本表中结点已经到达上限值,便将这个冲突的结点放入溢出缓冲表。

2.6 时间/空间局部性原理

2.6.1 系统局部性

在一个系统中，大部分程序都在进行过程处理，其会带来较大的消耗，如果能够减少，将会对程序起到优化作用，这也是程序性能优化需要考虑的地方。如何减少过程的处理历来是研究的重点，除了从算法上进行优化外，利用时间和空间的局部性也是一个重要的研究的方向。

一个从结构和功能上编写表现出良好且高效率的计算机程序更加倾向于良好的局部性。所谓的局部性，即计算机程序需要使用的数据项最好与其他最近使用过的数据项相邻，或者和最近自我使用过的数据项相邻近，这种邻近倾向，被称之为局部性原理^[38]。这个原理对计算机相关的设计都有十分重要的影响。

局部性通常表现在时间和空间上。对于一个拥有良好时间局部性的程序，在很短的周期内，已被用过的存储设备的地址较大概率再次被用。对于一个拥有良好空间局部性的程序，其使用存储设备的地址具有一种临近的状态，也即这次使用的存储器位置的临近位置很快可能被使用。

一般而言，对于拥有良好局部性的计算机程序会比局部性较差的程序运行的更快，这就是需要充分利用程序局部性的原因。存在一些量化的原则来评价一个程序，其如下：

- 同一个变量如果重复被引用，则在时间上表现出良好的局部性
- 在系统中如果存在步长为 n 的引用，那么 n 越小，空间局部性越好。
- 对于程序中的取指令操作，如果循环体小而且循环的次数多的话，那么它将表现出良好的局部性。

局部性的原理和存储器之间有很大的关系，因为局部性原理的产生就是通过引入高速的缓存存储器的缘故。存储器并不是简单的线性结构，实际上的存储器是具有不同容量、不同的访问速度，在性能上表现很大差异的层次结构，离 CPU 最近的存储设备保存着最常用的数据。存储器系统被设计成一个具有层次的结构，金字塔型结构原因是，为了平衡存储器性能和价格之间的关系，每一层表现出不同的容量、成本和访问的时间。如下图 2.3 所示。

从上往下存储器性能上表现较大差别，其层次结构设计的思想在于，对于 L 层更小、更快的存储设备，它是 $L+1$ 层的更大、更慢设备的缓存。在存储器的层次结构中，层次越高则越接近 CPU，表现出更快的速度，而且高层次存储器总是缓存来自较低层次存储器的数据。如果计算机程序需要的数据总能够从 $L1$ 和 $L2$ 层的高速缓存上获得数据，那么它表现出的性能将会比从 $L3$ 主存获取数据有高效的多。前者从 $L1$ 和 $L2$ 层的高速缓存上获得数据的程序相对而言表现出良好的局部性。在程序设计实现的时候，需要考虑更多的就是尽可能的从 $L1$ 和 $L2$ 层获取数据。

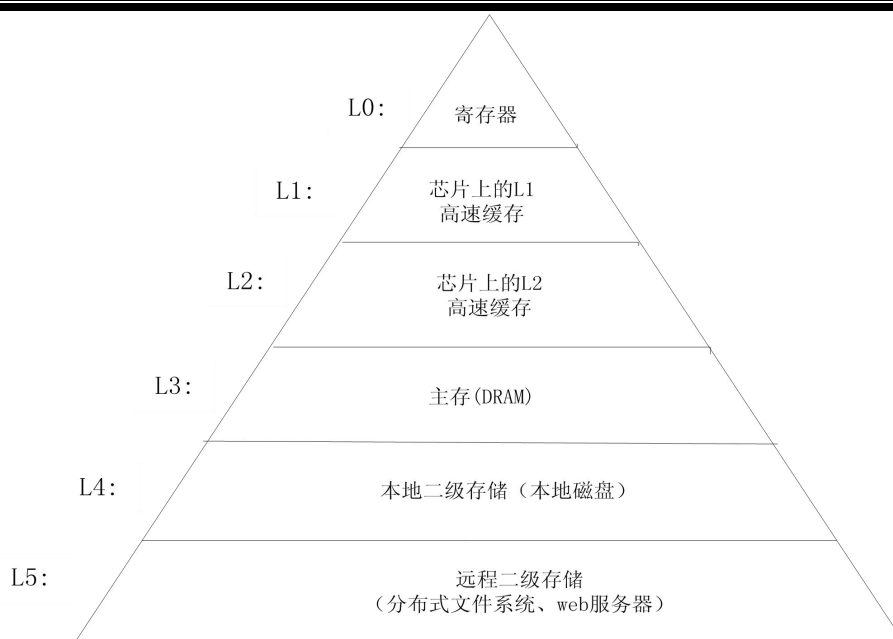


图2.3 存储器层次结构图

在计算机中有这样一个思想就是如果对存储器的层次结构理解深入，并且知道数据是如何在存储器结构的上上下下移动的，那么在编写程序的时候，就可以利用这样一个特性，就是让数据尽可能的处在最高层次、离 CPU 尽可能的近的地方，这样 CPU 获取它们的时候，就会表现的更快。据测试统计，如果 CPU 需要的数据在寄存器中，那么获取它们只需要零个时钟周期左右。但是如果是在高速缓存中的话，它的相对速度就会降低许多，需要 1 到 10 个时钟周期。如果在高速缓存中也没有找到数据的话，系统就会开始访问内存，数据在内存中，获取它们的速度将是 50 到 100 个时钟周期。这还是比较令人满意的。如果不幸的是在内存中也没有找到，而是存在了磁盘上，访问到它将需要 2 千万个时钟周期。可以看到对于不同的设备上的数据，访存差距非常大。

2.6.2 网络流量的局部性

研究者在对网络中的流量进行统计分析的时候，发现了网络数据流存在一个重要的现象，那就是它不是一种随机的现象，它表现出一种网络流量的时间和空间上的局部性。所谓网络流量的局部性，是指报文在传输过程中，它们并不是一个孤立的实体，它们在更高层次上表现出同一逻辑流信息，数据包之间存在较大的关联性，即存在这样的现象，当前处理的数据包，是由主机甲方发送给主机乙方的，下一个数据包很可能也是主机甲方发给乙方的，或者是主机乙方发给甲方的。这一点和大多数的 TCP 流会话是短暂存在相关^[26]。Caceres 等人研究表明，90%以上的 TCP 会话交换数据小于 10KB，持续不到几秒钟^[39,40,41]，而且会话的来到是一种满足泊松分布的现象^[26,42]。此外，网络流量的局部性还和协议的设计的方式和数据的传输方式存在相关性。

2.7 本章小结

本章首先从 WEB 防火墙性能测试评价的角度出发，对其中的几个指标进行简单的研究，接着重点钻研了本文将要使用的排队论理论，然后是对动态内存管理的详细研究。动态内存管理主要涉及两个方面，一是位图法，另一种是可利用空间表法。这是管理内存常用到的方法。最后对哈希冲突以及时间空间局部性原理进行了研究，其中重点阐述了网络流的局部性。

第3章 面向WEB防火墙排队论模型研究

3.1 引言

在WEB防火墙领域,提及的系统性能评价研究,相对比较简单。从数学模型角度对防火墙的研究主要是使用排队论的理论。一般的做法都是将防火墙看做一个排队系统进行研究分析,并不能很好的描述一个系统的内在的结构和相关关系。故本章将对WEB防火墙排队论模型进行深入分析,建立一种分层排队论的模型,然后对该分层排队论模型进一步深入研究。

3.2 /M/M/c/K 混合方式排队系统

/M/M/c/K 混合方式排队系统定义^[32]:在排队系统之中,总共存在K个位置以供排队等待,c个服务台可以进行相互独立的服务, $c \leq K$ 。当到达系统的顾客将全部的K个位置占满了,那么后续到达的顾客将不进入排队系统,当排队系统存在空闲位置的时候,后来的顾客就进行排队,等待服务。顾客是按照 $\lambda(>0)$ 的Poisson流的形式来到,对于它们的到来和服务是互相不相关,且服务时间也是互相不相关,而且时间分布函数是参数 μ 的负指数分布。

定理 3.1: 假定t时刻排队系统中顾客数量为 $N(t)$,其中只有一个服务台的服务强度为 $\rho = \frac{\lambda}{\mu}$ (λ 是单位时间内顾客的平均到来速率, μ 是一个服务台单位时间内的平均处理效率),系统稳定状态下,顾客数是j的概率为 $p_j = \lim_{t \rightarrow \infty} P\{N(t) = j\}$,则对于一切 ρ ,有如下等式:

$$p_j = \begin{cases} \frac{\rho^j}{j!} p_0, 1 \leq j < c \\ \frac{1}{c! c^{j-c}} \rho^j p_0, c \leq j \leq K \end{cases} \quad (3.1)$$

其中 $p_0 = [\sum_{n=0}^{c-1} \frac{1}{n!} \rho^n + \sum_{n=c}^K \frac{1}{c! c^{n-c}} \rho^n]^{-1}$ 。

3.3 WEB 防火墙系统分层排队论模型创建

在关于防火墙系统性能评价技术的研究中,更多的研究者主要是从一种数学理论的角度出发,研究系统各个参数之间的关系,抽象出一种理论模型,来说明防火墙系统。典型的方式就是将系统抽象成排队论模型。排队论模型将整个防火墙系统看做一个服务台或者说是多个服务台,来的数据包看做是消费者,或者说是顾客,消费者与服务台之间便形成了一个排队系统。排队理论模型是防火墙的一种抽象,它们主要研究的是各种性能指标和系统参数之间的一种高度概括的关系,比如说时延和数据流量到达速率、防

防火墙系统的处理以及系统的缓冲队列的长度之间的关系。起到了一定的指示作用，但是对于 WEB 防火墙系统的研究，还需要进一步深入和细化。

本文打算以 WEB 防火墙系统的分层结构为研究点，提出一种 WEB 防火墙系统的分层排队论模型。其模型结构如下图 3.1 所示。

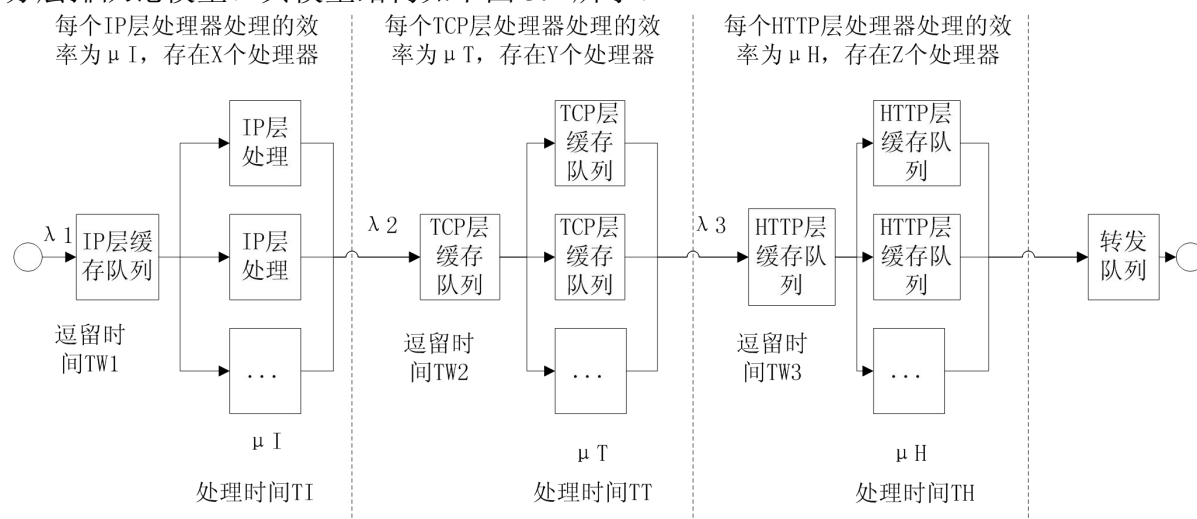


图3.1 WEB防火墙系统的分层排队论模型

对于 WEB 防火墙，其中涉及协议的三个层次，由上而下，分别是 HTTP 层、TCP 层以及 IP 层。从上图可以看出，WEB 防火墙系统的排队论模型，就是对这三个层次更加深入的研究，分层应用了排队理论进行研究。

对于第一层 IP 层的排队系统来说，输入是来自网络的数据流量，IP 层的输出的流量作为第二层 TCP 层排队系统的输入流量进行分析，TCP 层的输出流量则作为第三层 HTTP 层排队系统的输入流量。可见它们之间通过流量的关系紧密相关。

首先说明一下系统相关参数，其定义如下。

λ_1 是网络中进入 IP 层的流量速率。

μ_I 是 IP 层每个服务台的平均处理效率。

X 是 IP 层服务台总的个数。

K_I 是 IP 层的缓存队列的长度。

λ_2 是 IP 层的输出进入 TCP 层的流量速率。

μ_T 是 TCP 层每个服务台的平均处理效率。

Y 是 TCP 层服务台总的个数。

K_T 是 TCP 层的缓存队列的长度。

λ_3 是 TCP 层的输出进入 HTTP 层的流量速率。

μ_H 是 HTTP 层每个服务台的平均处理效率。

Z 是 HTTP 层服务台总的个数。

K_H 是 HTTP 层的缓存队列的长度。

对于每一层的处理都可以看做是 $/M/M/c/K$ 排队模型。

1) 对第一层 IP 层进行分析

在第一层排队系统中 $c = X, K = K_I$ 。

假定 IP 层的顾客是依照先来后到的方式进行处理, 设其到来时看到已经有 j 个顾客的稳定概率为 pi_j ,

$$pi_j = \begin{cases} \frac{\rho_I^j}{j!} pi_0, 1 \leq j < X, \\ \frac{1}{X! X^{j-X}} \rho_I^j pi_0, X \leq j \leq K_I \end{cases} \quad (3.2)$$

其中 $pi_0 = [\sum_{n=0}^{X-1} \frac{1}{n!} \rho_I^n + \sum_{n=X}^{K_I} \frac{1}{X! X^{n-X}} \rho_I^n]^{-1}$, $\rho_I = \frac{\lambda_1}{\mu_I}$, $j = 0, 1, 2, \dots, K_I$ 。

由于系统缓冲队列的限制, 当在系统排队队列中的顾客数达到了上限 K_I 的时候, 顾客便不能进入排队系统进行处理。由此可以知道不能进入排队系统的概率, 就是排队队列中含有顾客数为 K_I 的概率, 其为

$$p = pi_{K_I} = \frac{1}{X! X^{K_I-X}} \rho_I^{K_I} pi_0 \quad (3.3)$$

那么在单位时间中 IP 层平均损失的顾客数量为

$$\bar{\lambda}_{\text{损}} = \lambda p_{K_I} \quad (3.4)$$

由此可得到 IP 层吞吐量的计算公式是:

$$G_I = \lambda_1 \{1 - pi_{K_I}\} \quad (3.5)$$

即:

$$G_I = \lambda_1 \{1 - \frac{1}{X! X^{K_I-X}} (\frac{\lambda_1}{\mu_I})^{K_I} [\sum_{n=0}^{X-1} \frac{1}{n!} (\frac{\lambda_1}{\mu_I})^n + \sum_{n=X}^{K_I} \frac{1}{X! X^{n-X}} (\frac{\lambda_1}{\mu_I})^n]^{-1}\} \quad (3.6)$$

2) 对第二层 TCP 层进行分析

在第二层排队系统中 $c = Y, K = K_T$ 。

假定 TCP 层的顾客是依照先来后到的方式进行处理, 设其到来时看到已经有 j 个顾客的稳定概率为 pt_j ,

$$pt_j = \begin{cases} \frac{\rho_T^j}{j!} pt_0, 1 \leq j < Y, \\ \frac{1}{Y! Y^{j-Y}} \rho_T^j pt_0, Y \leq j \leq K_T \end{cases}, \quad (3.7)$$

其中 $pt_0 = [\sum_{n=0}^{Y-1} \frac{1}{n!} \rho_T^n + \sum_{n=Y}^{K_T} \frac{1}{Y! Y^{n-Y}} \rho_T^n]^{-1}$, $\rho_T = \frac{\lambda_2}{\mu_T}$ 。

在 TCP 层排队系统中, 顾客不能进入系统也是因为其缓冲队列达到了上限, 达到

上限的概率为系统中含有 K_T 个顾客的概率，即

$$p = pt_{K_T} = \frac{1}{Y!Y^{K_T-Y}} \rho_T^{K_T} pt_0 \quad (3.8)$$

由此可得到 TCP 层排队系统的系统吞吐量计算公式为是：

$$G_T = \lambda_2 \{1 - pt_{K_T}\} \quad (3.9)$$

即：

$$G_T = \lambda_2 \left\{ 1 - \frac{1}{Y!Y^{K_T-Y}} \left(\frac{\lambda_2}{\mu_T} \right)^{K_T} \left[\sum_{n=0}^{Y-1} \frac{1}{n!} \left(\frac{\lambda_2}{\mu_T} \right)^n + \sum_{n=Y}^{K_T} \frac{1}{Y!Y^{n-Y}} \left(\frac{\lambda_2}{\mu_T} \right)^n \right]^{-1} \right\} \quad (3.10)$$

3) 对第三层 HTTP 层进行分析

在第三层排队系统中 $c = Z, K = K_H$ 。

假定 HTTP 层的顾客是依照先来后到的方式进行处理，设其到来时看到已经有 j 个顾客的稳定概率为 ph_j ，

$$ph_j = \begin{cases} \frac{\rho_H^j}{j!} ph_0, 1 \leq j < Z, \\ \frac{1}{Z!Z^{j-Z}} \rho_H^j ph_0, Z \leq j \leq K_H \end{cases}, \quad (3.11)$$

其中 $ph_0 = \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \rho_H^n + \sum_{n=Z}^{K_H} \frac{1}{Z!Z^{n-Z}} \rho_H^n \right]^{-1}$ ， $\rho_H = \frac{\lambda_3}{\mu_H}$ 。

同理，在第三层排队系统中顾客不能进入原因也是因为其缓冲队列被占满的缘故。其概率为：

$$p = ph_{K_H} = \frac{1}{Z!Z^{K_H-Z}} \rho_H^{K_H} ph_0 \quad (3.12)$$

由此可得到 HTTP 层排队系统的系统吞吐量计算公式是：

$$G_H = \lambda_3 \{1 - ph_{K_H}\} \quad (3.13)$$

即：

$$G_H = \lambda_3 \left\{ 1 - \frac{1}{Z!Z^{K_H-Z}} \left(\frac{\lambda_3}{\mu_H} \right)^{K_H} \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \left(\frac{\lambda_3}{\mu_H} \right)^n + \sum_{n=Z}^{K_H} \frac{1}{Z!Z^{n-Z}} \left(\frac{\lambda_3}{\mu_H} \right)^n \right]^{-1} \right\} \quad (3.14)$$

3.4 WEB 防火墙系统吞吐量分析

本文最重要的目的在对研究系统吞吐量和系统 CPU 资源之间的关系，因此，下面将对吞吐量和系统 CPU 资源之间的关系进行深入研究。设定系统的吞吐量为 G ，其单位为 Mb/s。对系统吞吐量以及系统 CPU 资源分配的关系分析主要分成以下几步进行。

第一步：对 WEB 防火墙系统建立排队论模型，如图3.1所示；

第二步：使用上一节的分析，推导各个层次吞吐量公式 G_I 、 G_T 、 G_H ；

1) 对于第一层 IP 层

当 $\lambda_1 = X\mu_1$ 时，IP 层排队系统获取最大的吞吐量 G_I ，即下面公式所示：

$$G_I = X\mu_I \left(1 - \frac{1}{X! \sum_{n=0}^{X-1} \frac{1}{n!} X^{n-X} + K_I - X + 1}\right) \quad (3.15)$$

2) 第二层 TCP 层

第二层的排队系统的输入流量是由第一层输出流量构成，会有一定的流量的遗失。假定实际到达的流量比例为 α ，即实际到达的流量只有 αG_I ，也即 $\lambda_2 = \alpha G_I$ ，TCP 层的吞吐量公式为：

$$G_T = \alpha G_I \left\{1 - \frac{1}{Y! Y^{K_T-Y}} \left(\frac{\alpha G_I}{\mu_T}\right)^{K_T} \left[\sum_{n=0}^{Y-1} \frac{1}{n!} \left(\frac{\alpha G_I}{\mu_T}\right)^n + \sum_{n=Y}^{K_T} \frac{1}{Y! Y^{n-Y}} \left(\frac{\alpha G_I}{\mu_T}\right)^n\right]^{-1}\right\} \quad (3.16)$$

3) 第三层 HTTP 层

第三层的排队系统的输入流量是由第二层输出流量构成，也会有一定的流量的遗失。假定实际到达的流量比例为 β ，即实际到达的流量只有 βG_T ，也即 $\lambda_3 = \beta G_T$ ，HTTP 层的吞吐量公式为：

$$G_H = \beta G_T \left\{1 - \frac{1}{Z! Z^{K_H-Z}} \left(\frac{\beta G_T}{\mu_H}\right)^{K_H} \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \left(\frac{\beta G_T}{\mu_H}\right)^n + \sum_{n=Z}^{K_H} \frac{1}{Z! Z^{n-Z}} \left(\frac{\beta G_T}{\mu_H}\right)^n\right]^{-1}\right\} \quad (3.17)$$

第三步：确定系统总的可用 CPU 支持的处理进程或线程总数 N ，因为系统 CPU 资源限制了各个层次的服务台数。即： $X+Y+Z \leq N$ ；

第四步：根据第三步的结果，然后通过枚举 X 、 Y 、 Z 的值，求取第二步中的 G_I 、 G_T 、 G_H 值，其最小值，即 $\min\{G_I, G_T, G_H\}$ 为系统吞吐量 G 。通过这样的枚举计算，就能得到一系列的 G 的值，其实，是一张数据表，表中包含着 X 、 Y 、 Z 的值和系统吞吐量 G_I 、 G_T 、 G_H 以及 G 之间的关系，选取其中 G 的最大值，这个值就是系统的最大吞吐量 $G_{\max}$ ，此时的 X 、 Y 、 Z 可能就是一种最优的分配方式。

第五步：先对第四步得到的结果，进行理论分析，研究其规律性，看是否能够确定 X 、 Y 、 Z 即为最优。如果理论分析得不出结论，先降低 X 值，保持 Y 、 Z 不变，最小的 X 值使得 $G \geq G_{\max}$ ，记下此时的 X 值，同理对于 Y 和 Z ，也是这样的处理，直至找出最后最合适的 Y 和 Z 的值。此时的 X 、 Y 、 Z 即为最优分配的 X 、 Y 、 Z 。

第六步：也即是结论，系统最大吞吐量为 $G_{\max}$ ，系统资源的最优分配方式为当前的 X 、 Y 、 Z 值。

3.5 模型验证

分析理论计算值与实验值之间的关系——实例分析 1：

设定 $N=10$ ，IP 层的处理效率为 $\mu_I = 50Mb/s$ ，TCP 层的处理效率为 $\mu_T = 30Mb/s$ ，HTTP 层的处理效率为 $\mu_H = 15Mb/s$ ，取值 $K_I = K_T = K_H = 100$ 。假定 IP 层输出流量实际到达 TCP 层的比例为 $\alpha=0.8$ ，TCP 层输出流量实际达到 HTTP 层的比例为 $\beta=0.8$ 。

第一步：建立如图 3.1 排队论模型。

第二步：推导各个层次吞吐量公式。

1) 第一层 IP 层

将相应的数据信息带入公式 (3.15)，得到 IP 层排队系统获取最大的吞吐量 G_I ：

$$G_I = X\mu_I \left(1 - \frac{1}{X! \sum_{n=0}^{X-1} \frac{1}{n!} X^{n-X} + K_I - X + 1}\right)$$

2) 第二层 TCP 层

根据假定，实际到达的流量只有 $0.8G_I$ ，也即 $\lambda_2 = 0.8G_I$ ，带入公式 (3.16)，得到该层的吞吐量计算如下所示：

$$G_T = 0.8G_I \left\{1 - \frac{1}{Y!Y^{K_T-Y}} \left(\frac{0.8G_I}{\mu_T}\right)^{K_T} \left[\sum_{n=0}^{Y-1} \frac{1}{n!} \left(\frac{0.8G_I}{\mu_T}\right)^n + \sum_{n=Y}^{K_T} \frac{1}{Y!Y^{n-Y}} \left(\frac{0.8G_I}{\mu_T}\right)^n\right]^{-1}\right\}$$

3) 第三层 HTTP 层

根据假定，实际到达的流量只有 $0.8G_T$ ，也即 $\lambda_3 = 0.8G_T$ ，带入公式 (3.17)，得到 HTTP 层的吞吐量计算如下所示：

$$G_H = 0.8G_T \left\{1 - \frac{1}{Z!Z^{K_H-Z}} \left(\frac{0.8G_T}{\mu_H}\right)^{K_H} \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \left(\frac{0.8G_T}{\mu_H}\right)^n + \sum_{n=Z}^{K_H} \frac{1}{Z!Z^{n-Z}} \left(\frac{0.8G_T}{\mu_H}\right)^n\right]^{-1}\right\}$$

第三步：系统总的可用 CPU 支持的处理进程或线程总数 $N=10$ ，则 $X+Y+Z \leq 10$ ，因为相对处理强度为 $\frac{1}{\mu_I} < \frac{\alpha}{\mu_T} < \frac{\alpha\beta}{\mu_H}$ ，所以分配时候，还存在一个条件 $X \leq Y \leq Z$ 。

第四步：枚举 X 、 Y 、 Z 的值，求取第二步中的 G_I 、 G_T 、 G_H 值，其最小值，即 $\min\{G_I, G_T, G_H\}$ 为系统吞吐量 G 。

模型计算的数据如下表 3.1 所示：

表 3.1 理论计算中 X 、 Y 、 Z 的取值与吞吐量关系 单位：Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	8	49.5	30	25	25
1	2	7	49.5	39.6	31.68	31.68
1	3	6	49.5	39.6	31.68	31.68
1	4	5	49.5	39.6	31.68	31.68
2	2	6	99.0	60.0	48.0	48.0
2	3	5	99.0	79.20	63.36	63.36
2	4	4	99.0	79.2	59.98	59.98
3	3	4	148.5	90	60	60

可以从上表看出，当 $X=2$ ， $Y=3$ ， $Z=5$ ，WEB 防火墙系统取得吞吐量的最大值为 63.36Mb/s。至于此时的 X 、 Y 、 Z 是不是最优的还需要进一步分析。

第五步：先对 X 的值进行分析。当 X 减少为 1 的时候（即 $X=1$ ， $Y=4$ ， $Z=5$ ） G_I 只

有 49.5Mb/s, 比当前 G_I 小, 可见此时的 X 已经是最小值了。其次, 对 Y 进行分析。当 Y 减小为 2 的时候 (即 X=2, Y=2, Z=6), 可以见到 G_T 比当前要小, 所以 Y 也是最小取值。最后分析一下 Z, 与 X=2, Y=4, Z=4 进行对比, 可以看见当 Z 减小的时候, G_Z 减小了。可见当前 Z 的值也是最小值。

第六步: 得出结论。系统最大吞吐量为 63.36Mb/s, 当前服务台分配为 X=2, Y=3, Z=5, 此分配情况下系统达到最优。

实验验证:

实验程序采用 linux 下 C 语言编写。模拟 IP 报文的平均长度为 500B, 每个 TCP 报文由四个 IP 报文重组而成, 解析完成的 TCP 数据报文放入 HTTP 数据分析队列。IP 层的处理效率为 $\mu_I = 50Mb/s$, TCP 层的处理效率为 $\mu_T = 30Mb/s$, HTTP 层的处理效率为 $\mu_H = 15Mb/s$, 取值 $K_I = K_T = K_H = 100$ 。其中 TCP 层和 HTTP 层实际到达的流量比例 α 、 β 皆设为 0.8。

程序测试结果如下表 3.2 所示:

表 3.2 实验中的 X、Y、Z 的取值与吞吐量关系 单位: Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	8	49.86	26.16	20.92	20.92
1	2	7	49.87	39.92	31.92	31.92
1	3	6	49.87	39.88	31.92	31.92
1	4	5	49.87	39.89	31.92	31.92
2	2	6	99.69	52.48	41.98	41.98
2	3	5	99.70	78.60	62.90	62.90
2	4	4	99.68	79.72	54.06	54.06
3	3	4	149.45	78.64	55.70	55.70

从实验测试结果来看, 系统达到最优的状态也是服务台分配为 X=2, Y=3, Z=5, 此时系统的吞吐量为 62.90Mb/s。理论相对实际误差为 0.73%。

分析理论计算值与实验值之间的关系——实例分析 2:

设定 N=9, IP 层的处理效率为 $\mu_I = 52Mb/s$, TCP 层的处理效率为 $\mu_T = 32Mb/s$, HTTP 层的处理效率为 $\mu_H = 18Mb/s$, 取值 $K_I = 1000, K_T = 400, K_H = 100$ 。假定 IP 层输出流量实际到达 TCP 层的比例为 $\alpha=0.85$, TCP 层输出流量实际达到 HTTP 层的比例为 $\beta=0.8$ 。

第一步：建立如图 3.1 排队论模型。

第二步：推导各个层次吞吐量公式。

1) 第一层 IP 层

将相应的数据信息带入公式 (3.15)，得到 IP 层排队系统获取最大的吞吐量 G_I ：

$$G_I = X\mu_I \left(1 - \frac{1}{X! \sum_{n=0}^{X-1} \frac{1}{n!} X^{n-X} + K_I - X + 1}\right)$$

2) 第二层 TCP 层

根据假设，实际到达的流量只有 $0.85G_I$ ，也即 $\lambda_2 = 0.85G_I$ ，带入公式 (3.16)，得到该层的吞吐量计算如下所示：

$$G_T = 0.85G_I \left\{1 - \frac{1}{Y!Y^{K_T-Y}} \left(\frac{0.85G_I}{\mu_T}\right)^{K_T} \left[\sum_{n=0}^{Y-1} \frac{1}{n!} \left(\frac{0.85G_I}{\mu_T}\right)^n + \sum_{n=Y}^{K_T} \frac{1}{Y!Y^{n-Y}} \left(\frac{0.85G_I}{\mu_T}\right)^n\right]^{-1}\right\}$$

3) 第三层 HTTP 层

根据假定，实际到达的流量只有 $0.8G_T$ ，也即 $\lambda_3 = 0.8G_T$ ，带入公式 (3.17)，得到 HTTP 层的吞吐量计算如下所示：

$$G_H = 0.8G_T \left\{1 - \frac{1}{Z!Z^{K_H-Z}} \left(\frac{0.8G_T}{\mu_H}\right)^{K_H} \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \left(\frac{0.8G_T}{\mu_H}\right)^n + \sum_{n=Z}^{K_H} \frac{1}{Z!Z^{n-Z}} \left(\frac{0.8G_T}{\mu_H}\right)^n\right]^{-1}\right\}$$

第三步：系统总的可用 CPU 支持的处理进程或线程总数 $N=9$ ，则 $X+Y+Z \leq 9$ ，因为相对处理的强度为 $\frac{1}{\mu_I} < \frac{\alpha}{\mu_T} < \frac{\alpha\beta}{\mu_H}$ ，所以分配时候，还存在一个条件 $X \leq Y \leq Z$ 。

第四步：枚举 X、Y、Z 的值，求取第二步中的 G_I 、 G_T 、 G_H 值，其最小值，即 $\min\{G_I, G_T, G_H\}$ 为系统吞吐量 G。

模型计算的数据如下表 3.3 所示：

表 3.3 理论计算中 X、Y、Z 的取值与吞吐量关系 单位：Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	7	51.95	32.0	25.6	25.6
1	2	6	51.95	44.15	35.32	35.32
1	3	5	51.95	44.15	35.32	35.32
1	4	4	51.95	44.15	35.32	35.32
2	2	5	103.90	64.0	51.2	51.2
2	3	4	103.90	88.31	70.41	70.41
3	3	3	155.84	96.0	54.0	54.0

可以从上表看出，当 $X=2$ ， $Y=3$ ， $Z=4$ ，WEB 防火墙系统取得吞吐量的最大值为 70.41Mb/s。至于此时的 X、Y、Z 是不是最优的还需要进一步分析。

第五步：先对 X 的值进行分析。当 X 减少为 1 的时候（即 $X=1$ ， $Y=4$ ， $Z=4$ ） G_I 只

有 51.95Mb/s, 比当前 G_I 小, 可见此时的 X 已经是最小值了。其次, 对 Y 进行分析。当 Y 减小为 2 的时候 (即 $X=2, Y=2, Z=5$), 可以见到 G_T 比当前要小 ($64.0 < 88.31$), 所以 Y 也是最小取值。最后分析一下 Z , 与 $X=3, Y=3, Z=3$ 进行对比, 可以看见当 Z 减小的时候, G_Z 减小了 ($54.0 < 70.41$)。可见当前 Z 的值也是最小值。

第六步: 得出结论。系统最大吞吐量为 70.41Mb/s, 当前服务台分配为 $X=2, Y=3, Z=4$, 此分配情况下系统达到最优。

实验验证:

模拟 IP 报文的平均长度为 500B, 每个 TCP 报文由四个 IP 数据包重组而成, 解析完成的 TCP 数据报文放入 HTTP 数据分析队列。IP 层的处理效率为 $\mu_I = 52Mb/s$, TCP 层的处理效率为 $\mu_T = 32Mb/s$, HTTP 层的处理效率为 $\mu_H = 18Mb/s$, 取值 $K_I = 1000, K_T = 400, K_H = 100$ 。其中 TCP 层和 HTTP 层实际到达的流量比例 α 、 β 分别设为 0.85 和 0.8。

程序测试结果如下表 3.4 所示:

表 3.4 实验中的 X 、 Y 、 Z 的取值与吞吐量关系 单位: Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	7	52.48	27.79	22.24	22.24
1	2	6	52.48	44.60	35.67	35.67
1	3	5	52.49	44.63	35.70	35.70
1	4	4	52.49	44.61	35.67	35.67
2	2	5	104.94	55.73	44.56	44.56
2	3	4	104.95	83.43	66.74	66.74
3	3	3	157.26	83.96	53.98	53.98

从实验测试结果来看, 系统达到最优的状态也是服务台分配为 $X=2, Y=3, Z=4$, 此时系统的吞吐量为 66.74Mb/s。理论相对实际误差为 5.50%。

分析理论计算值与实验值之间的关系——实例分析 3:

设定 $N=9$, IP 层的处理效率为 $\mu_I = 55Mb/s$, TCP 层的处理效率为 $\mu_T = 36Mb/s$, HTTP 层的处理效率为 $\mu_H = 26Mb/s$, 取值 $K_I = 2000, K_T = 600, K_H = 300$ 。假定 IP 层输出流量实际到达 TCP 层的比例为 $\alpha=0.88$, TCP 层输出流量实际达到 HTTP 层的比例为 $\beta=0.85$ 。

第一步: 建立如图 3.1 排队论模型。

第二步：推导各个层次吞吐量公式。

1) 第一层 IP 层

将相应的数据信息带入公式 (3.15)，得到 IP 层排队系统获取最大的吞吐量 G_I ：

$$G_I = X\mu_I \left(1 - \frac{1}{X! \sum_{n=0}^{X-1} \frac{1}{n!} X^{n-X} + K_I - X + 1}\right)$$

2) 第二层 TCP 层

根据假设，实际到达的流量只有 $0.88G_I$ ，也即 $\lambda_2 = 0.88G_I$ 。带入公式 (3.16)，得到该层的吞吐量计算如下所示：

$$G_T = 0.88G_I \left\{1 - \frac{1}{Y!Y^{K_T-Y}} \left(\frac{0.88G_I}{\mu_T}\right)^{K_T} \left[\sum_{n=0}^{Y-1} \frac{1}{n!} \left(\frac{0.88G_I}{\mu_T}\right)^n + \sum_{n=Y}^{K_T} \frac{1}{Y!Y^{n-Y}} \left(\frac{0.88G_I}{\mu_T}\right)^n\right]^{-1}\right\}$$

3) 第三层 HTTP 层

根据假定，实际到达的流量只有 $0.85G_T$ ，也即 $\lambda_3 = 0.85G_T$ 。带入公式 (3.17)，得到 HTTP 层的吞吐量计算如下所示：

$$G_H = 0.85G_T \left\{1 - \frac{1}{Z!Z^{K_H-Z}} \left(\frac{0.85G_T}{\mu_H}\right)^{K_H} \left[\sum_{n=0}^{Z-1} \frac{1}{n!} \left(\frac{0.85G_T}{\mu_H}\right)^n + \sum_{n=Z}^{K_H} \frac{1}{Z!Z^{n-Z}} \left(\frac{0.85G_T}{\mu_H}\right)^n\right]^{-1}\right\}$$

第三步：系统总的可用 CPU 支持的处理进程或线程总数 $N=9$ ，则 $X+Y+Z \leq 9$ ，因为相对处理的强度为 $\frac{1}{\mu_I} < \frac{\alpha}{\mu_T} < \frac{\alpha\beta}{\mu_H}$ ，所以分配时候，还存在一个条件 $X \leq Y \leq Z$ 。

第四步：枚举 X 、 Y 、 Z 的值，求取第二步中的 G_I 、 G_T 、 G_H 值，其最小值，即 $\min\{G_I, G_T, G_H\}$ 为系统吞吐量 G 。

模型计算的数据如下表 3.5 所示：

表 3.5 理论计算中 X 、 Y 、 Z 的取值与吞吐量关系 单位：Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	7	54.97	36.00	30.60	30.60
1	2	6	54.97	48.37	41.12	41.12
1	3	5	54.97	48.37	41.12	41.12
1	4	4	54.97	48.37	41.12	41.12
2	2	5	109.95	72.00	61.20	61.20
2	3	4	109.95	96.75	82.24	82.24
3	3	3	164.92	108.00	78.00	78.00

可以从上表看出，当 $X=2$ ， $Y=3$ ， $Z=4$ ，WEB 防火墙系统取得吞吐量的最大值为 82.24Mb/s。至于此时的 X 、 Y 、 Z 是不是最优的还需要进一步分析。

第五步：先对 X 的值进行分析。当 X 减少为 1 的时候（即 $X=1$ ， $Y=4$ ， $Z=4$ ） G_I 只有 54.97Mb/s，比当前 G_I （109.95）小，可见此时的 X 已经是最小值了。其次，对 Y 进

行分析。当Y减小为2的时候（即 $X=2, Y=2, Z=5$ ），可以见到 G_T 比当前要小（ $72.00 < 96.75$ ），所以Y也是最小取值。最后分析一下Z，与 $X=3, Y=3, Z=3$ 进行对比，可以看见当Z减小的时候， G_Z 减小了（ $78.00 < 82.24$ ）。可见当前Z的值也是最小值。

第六步：得出结论。系统最大吞吐量为82.24Mb/s，当前服务台分配为 $X=2, Y=3, Z=4$ ，此分配情况下系统达到最优。

实验验证：

模拟IP报文的平均长度为500B，每个TCP报文由四个IP数据包重组而成，解析完成的TCP数据报文放入HTTP数据分析队列。IP层的处理效率为 $\mu_I = 55Mb/s$ ，TCP层的处理效率为 $\mu_T = 36Mb/s$ ，HTTP层的处理效率为 $\mu_H = 26Mb/s$ ，取值 $K_I = 2000, K_T = 600, K_H = 300$ 。其中TCP层和HTTP层实际到达的流量比例 α 、 β 分别设为0.88和0.85。

程序测试结果如下表3.6所示：

表3.6 实验中的X、Y、Z的取值与吞吐量关系 单位：Mb/s

X	Y	Z	G_I	G_T	G_H	$G_{\min}$
1	1	7	55.38	31.41	26.70	26.70
1	2	6	55.40	48.75	41.45	41.45
1	3	5	55.40	48.75	41.45	41.45
1	4	4	55.40	48.75	41.44	41.44
2	2	5	110.74	63.30	53.80	53.80
2	3	4	110.74	94.15	80.06	80.06
3	3	3	166.01	94.15	77.97	77.97

从实验测试结果来看，系统达到最优的状态也是服务台分配为 $X=2, Y=3, Z=4$ ，此时系统的吞吐量为80.06Mb/s。理论相对实际误差为2.72%。

总结：从上面的实验结果和理论分析结果看来，对于系统的最优服务台分配理论和实验结果一致，但是对于WEB防火墙系统最优情况下的吞吐量看来有一定的误差，实际模拟程序的吞吐量要比理论值低，误差范围在6%以内，可以将理论用于指导实际资源分配和WEB防火墙系统的预估。

3.6 本章小结

本章主要使用数学模型中排队论理论，深层次分析了WEB防火墙。将WEB防火墙按照TCP/IP协议模型分层了三个层次结构，并对其建立了一个分层排队论研究模型。在此模型的基础之上，对WEB防火墙系统的各层吞吐量以及系统整体吞吐量和计算资

源之间的关系进行了深入的研究。得到的结论就是计算资源在 WEB 防火墙中各个层次的最优分配, 以及防火墙系统最大的吞吐量的值。在理论分析之外, 并且进行了实验验证, 说明了理论的正确性。

第4章 一种面向WEB防火墙性能优化方案

4.1 引言

一般要使编写的程序表现出高效性，需要从两个方面进行^[38]。第一，在数据结构和算法，进行最好的选择；第二，编写能够让编译器进行有效优化的源代码，并最终转化成高效的可执行程序。

在本章，将对系统的优化技术进行深入的理解和研究，考虑的是提高代码的处理效率，提高它的处理性能，从而达到优化系统的目的。主要从两个方向入手研究，一是动态内存管理中的内存池技术，另一个则是哈希表的冲突解决技术。

4.2 WEB 防火墙优化方案设计

4.2.1 内存池优化技术

由上文相关理论的研究可以知道一个充分利用局部性的程序，在性能上表现会更好。因此，针对传统的内存池链表的实现技术，目前研究者利用时间和空间局部性对其进行优化^[25]，下文将说明一下他的优化的思想。

针对链表的内存池结构的缺点，在局部性原理上表现的较差，被改进为使用内存下标索引的方式，设计改进的数据结构如下图4.1所示：

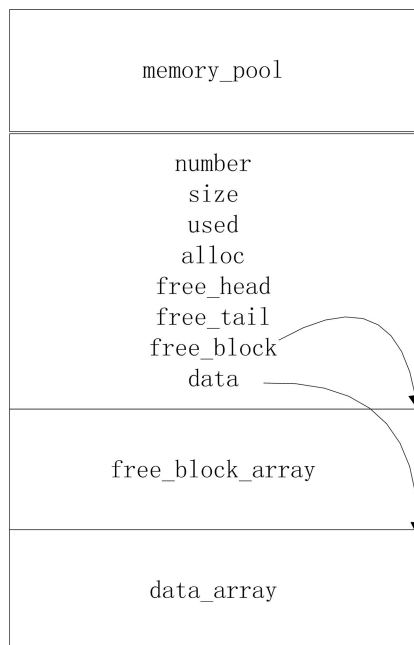


图 4.1 内存池结构图

先确定相关的初始化状态，相关参数像 `used`、`alloc`、`free_head`、`free_tail` 等置为 0，`free_block` 指向的空间置为-1。基于此前提，在内存申请的时候，相关算法是^[25]：

比较 `used` 和 `alloc` 的大小，若 `used` 比 `alloc` 大，可知索引队列中有可以使用的空闲块，那么从队首获得一个空闲块的索引值返回，`alloc` 的值加 1。否则的话，对 `used` 和 `number` 进行比较，若 `used` 比 `number` 的值大，那么无法从内存池中获取内存块，返回失败。若 `used` 比 `number` 的值小，那么返回 `used` 的值，申请者可以通过此索引，使用相应的空闲空间，然后 `used` 和 `alloc` 都需要加 1。

内存释放的时候使用的算法是^[25]：

释放过程比较简单，传入的值是内存空间的索引，先对索引的值进行合法性的分析，然后分析该内存空间可曾已经被释放过，防止被多次处理，如果检查都合法，便将该索引值插入队列的末尾，同时将该索引标识的内存空间置为未被使用。最后，将 `alloc` 的值进行减 1。

在研究上述算法的时候发现，该算法存在一个可以优化的地方。可以看到该算法，在存储内存块索引下标的时候使用的是队列，队列的时间局部性和空间局部性相对于栈来说要差一些。这是由队列和栈的性质决定的。队列的性质在于先进先出，进入队列的最新元素不能立刻被使用，而相反，栈就存在这样的优点，栈的性质在于先进后出，存入栈顶的元素，会在不久的将来，立刻被使用。相比较而言，可以知道，如果在这个地方，进行改进的话，性能将会得到提升。算法改进设计如下图 4.2 所示。

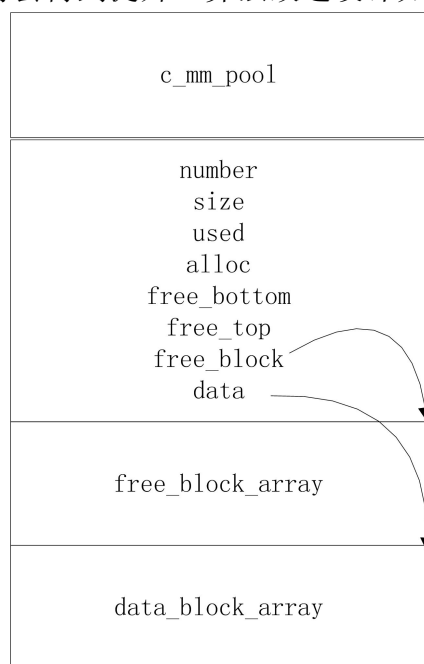


图4.2 改进后内存池结构图

优化后的算法，并没有对内存池的结构进行多大的改变。它相对与原算法最大的改变就是数据的插入和取出，改进后的内存池的结构体 C 定义为：

```

typedef struct c_mm_pool
{
    int number;        /*内存块个数*/
    int size;          /*内存块大小*/
    int used;           /*从未使用过内存块的起始位置*/
    int alloc;          /*正在使用的内存块个数*/
    int free_top;       /*栈顶*/
    int *free_block;    /*指向存储索引栈指针*/
    char *data;         /*指向内存块指针*/
}CMM_POOL;

```

改进后的算法描述如下。

相关参数初始化，used、alloc、free_top 等赋值为 0。

内存分配算法描述如下：

先比较 used 和 alloc 的大小，若 used 比 alloc 大，可知空闲空间栈中存在可以使用的索引，那么从栈顶获得一个空闲空间的索引返回给申请者，alloc 的值加 1。否则的话，对 used 和 number 进行比较，若 used 比 number 的值大，那么无法从内存池中获取内存空间，返回失败。若 used 比 number 的值小，那么返回 used 的值，申请者可以通过此下标，使用对应的空闲块，然后 used 和 alloc 都需要加 1。

其 C 代码描述如下图 4.3 所示：

```

int index;
/* 如果栈中有可以使用的空间，直接从栈中取出空闲块索引 */
if(mm_pool->used > mm_pool->alloc)
{
    mm_pool->free_top = mm_pool->free_top - 1;
    /* 将栈顶的元素弹出 */
    index = mm_pool->free_block[mm_pool->free_top];

    mm_pool->alloc ++;
    return index;
}
/* 如果已经没有使用的空间，返回失败 */
if(mm_pool->used >= mm_pool->number)
{
    return -1;
}
/* 当已经使用过的内存块不够用的时候，取从未使用的块 */
index = mm_pool->used;
/* 未被使用的空闲块往后移位加 1 */
mm_pool->used ++;
mm_pool->alloc ++;
return index;

```

图4.3 改进分配的 C 描述

内存空间回收算法描述如下：

当内存空间使用完毕，归还内存池时，传的参数是索引号，该算法会对传进入的参数的合法性进行判断，然后判断是否有可释放的内存，如果一切都合法，便将索引插入栈顶。已经使用的空闲块减 1。

其 C 代码描述如下图 4.4 所示：

```
/* 传入的参数存在问题，返回失败 */
if(index < 0 || index >= mm_pool->number)
    return -1;
/* 没有可以释放的内存，返回失败 */
if(mm_pool->alloc <= 0)
    return -1;
/* 将需要的释放的索引下标，插入栈顶，栈顶下移 */
mm_pool->free_block[mm_pool->free_top] = index;
mm_pool->free_top = mm_pool->free_top + 1;
/* 占用的空闲块减 1 */
mm_pool->alloc --;
return 0;
```

图4.4 改进释放的 C 描述

后续将采用实验方法证明改进思想的可行。

4.2.2 哈希表查找优化技术

在上文中已经提到有关哈希冲突解决的办法，其中最常使用的便是链式解决冲突法。链式解决冲突的优点在于它能够很好的解决冲突，因为对于每一个冲突的结点，它都能够很好的链入到哈希链表中。但是，这却也带来了一个严重的缺点，那就是链表的长度变长，查找的时候，因为搜索链表的缘故，会产生较大的时延。

传统的链式解决冲突法，被引用到网络数据流的哈希处理当中，但是它并没有有效的利用网络数据流的局部性的特性，从而可以从这个地方进行优化。西安交通大学的研究者提出了一种利用 IP 流局部性的相关优化方法^[28]，以及长沙理工大学和华中科技大学的研究者提出了一种利用网络流局部性的相关优化方法^[24]。他们想法的共同点就是利用网络中数据到达的局部性原理，提出了针对链式冲突有效的优化的办法。便是 MTF（Move to front）或者 MTH（Move to head）方法。所谓 MTF 或者 MTH，其原理性是一致的，就是将当前处理的链表中的结点，移动到链表的表头，哈希表中这个结点之前的结点将相应的后退一个位置。其原理工作图如下图 4.5 所示。

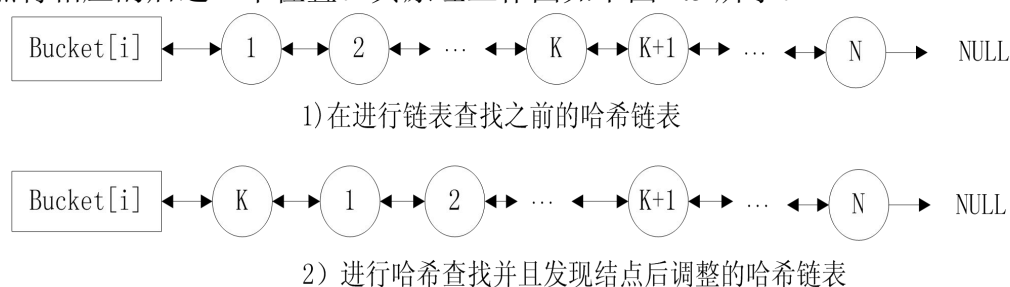


图4.5 MTF 算法工作原理

仔细观察上图 4.5 中的 1) 和 2)，图 4.5 中的 1) 是在进行结点查找之前的状态，K 处在链表中的某一处，但是经过一轮查找之后，找到了结点 K，MTF 算法并不是如传统算法一般，简单的返回处理，而是将结点 K 移动到链表的表头，重新组织哈希链，K 之前的结点会相应的后退一个结点。如图 4.5 中的 2) 所示。经过如此处理之后，所有比较活跃的结点会靠近链表的表头，在搜索查找的时候，查找的链表的平均长度就会相应的变短，这样查找的效率就变得更加的高效。

在研究上面处理方法的时候，发现上面的方法，存在一点缺点，那就是结点之间的频繁移动的处理，因为对于每一个结点的到来，它都会相应的调整结点之间的顺序，这样在处理的时候，会相对于耗时一些，如果利用空间换取时间的思想，引用 CACHE 指针操作，或许会相对变得简单，而效率会更加的高效。因此对于这一点，本文做出如下的改进。改进后的算法（简称为 CACHE）原理如下图 4.6 所示。

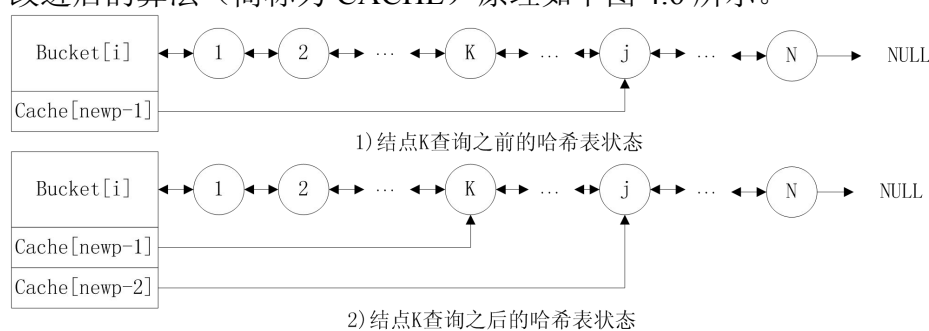


图4.6 CACHE 算法工作原理

本文改进的地方在于，不再是将通过结点移动来达到利用网络流局部性的原理。而是通过增设几个缓存指针，来达到利用网络流的局部性的原理。增设缓存指针的目的相对于 MTF 算法最大的优势在于减少指针的移动，以及因为指针移动带来的影响。可以对比上图 4.6 中的 1) 和 2) 来说明 CACHE 算法带来的变动。图 4.6 中的 1) 是结点 K 到来，但是在查询更新链表之前的状态，最新的 CACHE 指针指向 j，经过一轮查询之后，发现最新到达到的结点不是 j，而是 K 了，此时，会对数据进行更新，将最新的结点 K 的地址压入到栈中，也即保存到 CACHE 数组中，成为最新的缓存结点，而 j 则会变成历史缓存，相对最新的结点为第二新。需要说明的是，这里使用栈，因为在前文已经证明过，栈的局部性要好。

改进后的算法描述如下：

1. 对新来数据使用哈希计算，得到其对应地址；
2. 对缓存数组进行查找，如果查到，更新缓存信息，将当前结点设为最新的结点；之前栈中存储的索引值，直接丢弃；返回；否则转入步骤 3；
3. 进行链表查找，查看是否已经存在需要查询的数据值；如果不存在进行下一步；如果存在，转至步骤 5；

4. 采用头插入法，将最新的数据链入链表；返回；
5. 修改基表结构体信息，将最新的结点的地址压入栈中保存，以便下次查找；返回；
6. 算法结束

后续将采用实验方法证明改进思想可行。

4.3 实验及结果分析

4.3.1 内存池优化实验

本实验使用的测试环境为：操作系统 linux 64 位系统，内核版本 2.6.32，cpu 核数为 6 核，频率为 1200MHz，16G 内存。gcc 版本为 4.4.5。

本实验打算根据系统中结构体所占用空间的情况，来测试程序的性能。选择的测试的申请的空间大小为 88、128 和 1024 字节。88 字节是系统中结构体所占内存空间的大小，也是为了避免都是 2 的整数次方内存。本次实验参加对比的算法有三个，一个是改进前的队列模式的内存空间申请和释放，一个是改进后的栈模式的内存空间的申请和释放，一个是直接使用 malloc 和 free 函数的相关算法。

实验步骤如下：

- 1) 测试 1 百万次内存空间申请和释放的情况下，记录各个算法使用的时间情况。
- 2) 测试 2 百万次内存空间申请和释放的情况下，记录各个算法使用的时间情况。
- 3) 测试 5 百万次内存空间申请和释放的情况下，记录各个算法使用的时间情况。
- 4) 测试 1 千万次内存空间申请和释放的情况下，记录各个算法使用的时间情况。
- 5) 对各个算法使用的时间情况进行分析。

结果与分析：

1、处理 88 字节消耗的时间的测试结果

测试实验的结果如下表 4.1 所示。

表 4.1 处理 88 字节使用的时间对比 单位：微秒

实验次数（百万）	1	2	5	10
malloc/free	18130	39020	99247	198509
队列模式内存池	15543	31876	70361	144543
栈模式内存池	12232	25607	50754	100303

转化为图显示如下图 4.7 所示。

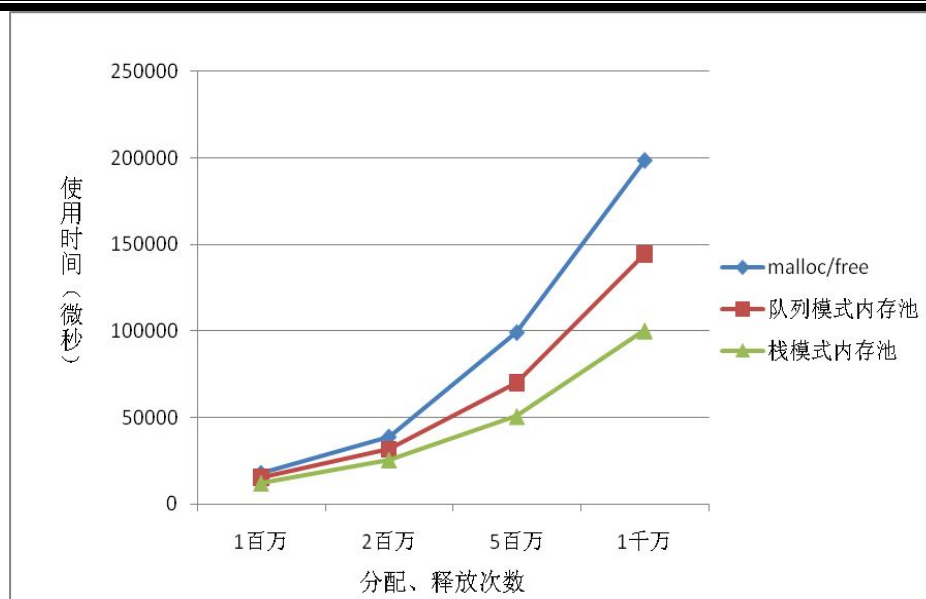


图 4.7 处理 88 字节的时间消耗对比

2、处理 128 字节消耗的时间的测试结果

测试实验的结果如下表 4.2 所示。

表 4.2 处理 128 字节使用的时间对比 单位：微秒

实验次数（百万）	1	2	5	10
malloc/free	48940	99249	251588	550593
队列模式内存池	19726	30917	71438	135138
栈模式内存池	12311	24777	49662	95157

转化为图显示如下图 4.8 所示。

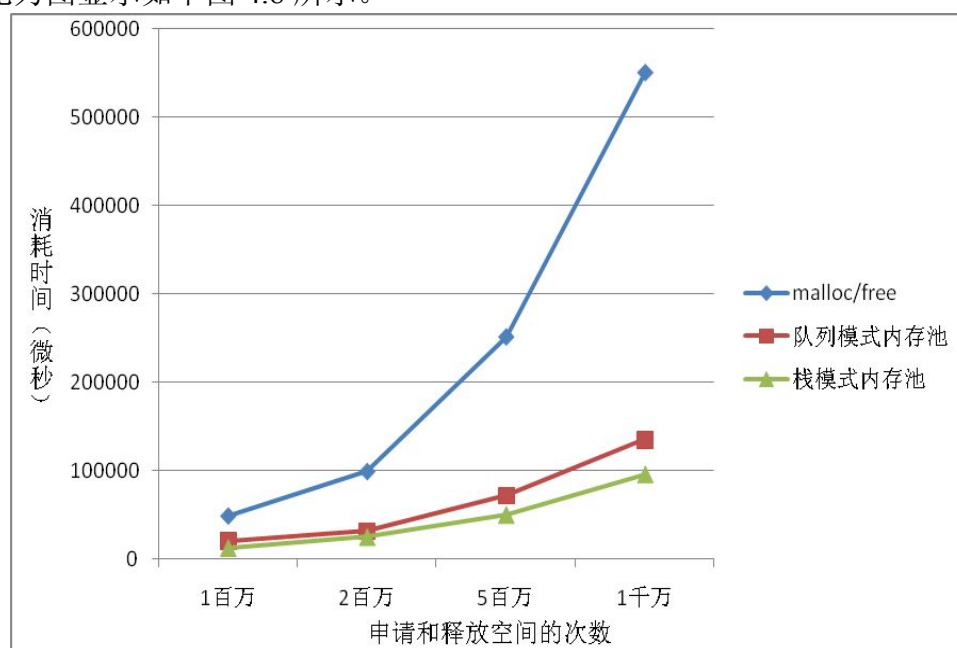


图 4.8 处理 128 字节时间消耗对比

3、处理 1024 字节消耗的时间的测试结果

测试实验的结果如下表 4.3 所示。

表 4.3 处理 1024 字节使用的时间对比 单位：微秒

实验次数（百万）	1	2	5	10
malloc/free	179875	481274	1464482	2879933
队列模式内存池	15495	29983	77067	145062
栈模式内存池	12456	25009	50218	102933

转化为图显示如下图 4.9 所示。

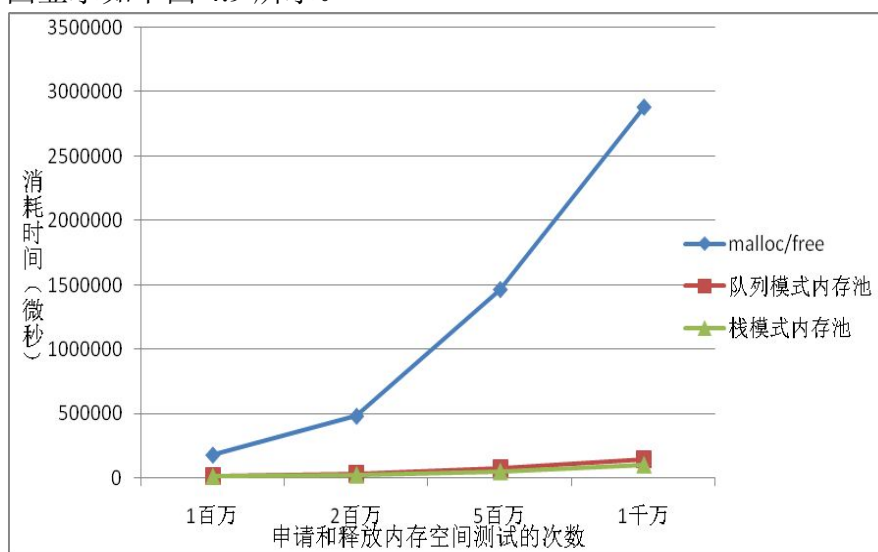


图 4.9 处理 1024 字节时间消耗对比

为了使对内存池的时间的对比效果更加的明显，只使用了它们两个的实验数据进行对比，生成了如下图 4.10 所示。

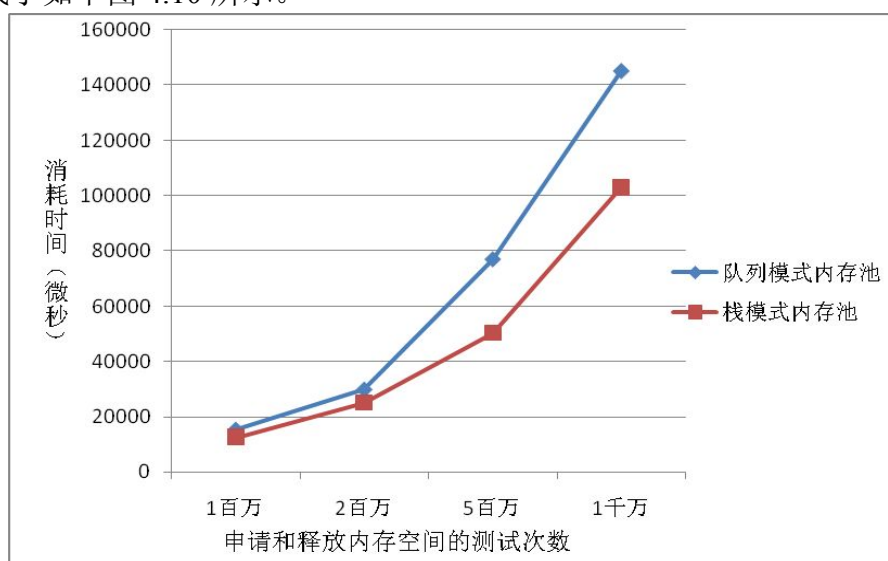


图 4.10 改进前后处理 1024 字节时间消耗对比

经过实验对比,从图 4.7、图 4.8、图 4.9、图 4.10 可以看出,随着申请和释放空间的次数的增加,消耗的时间也呈现一种递增的趋势。采用内存池技术,明显会比使用 malloc 和 free 函数,带来明显的效率提升。通过对比分析,内存池改进前和改进后的实验数据,也明显的可以看到,经过队列到栈的改变,系统的局部性效果得到了进一步的提高,从而使程序的性能得到了较好的改变。总体来说,改进后的方法,在处理效率上得到了 20%~30%的提升。

4.3.2 哈希表查找优化实验

本实验使用的测试环境为:操作系统 linux 64 位系统,内核版本 2.6.32,cpu 核数为 6 核,频率为 1200MHz,16G 内存。gcc 版本为 4.4.5。

本实验采取的是模拟网络流的仿真实验,开发实现使用的 linux 下 c 语言。模拟的前提假设是网络中 TCP 流的个数,每秒新增加的速度是泊松分布的,而且每个 TCP 流保持的时间是指数分布。

假设网络中平均每秒新增加 TCP 流的个数为 5 万个,每个 TCP 流平均的持续时间为 4 秒钟,每个 TCP 流平均含有数据包的个数为 128 个。

本次实验参加对比的算法有三个,一个是正常的链式哈希冲突解决办法的算法,每一个新加入的冲突结点直接连到链表的表头(称为 NORMAL 方法),一个是利用网络流的局部性改进了的链式哈希冲突解决方法,将当前处理的结点移动到表头的方法(称为 MTF 方法),一个是本文介绍和改进的算法,类似利用 CACHE 缓存的办法,但是减少指针的移动(称为 CACHE 方法)。在实验中测试存储的是每一个算法处理每一万条流的时间和,根据时间的大小判断算法的优劣。

实验方法和步骤如下进行:

1) 设置表的大小为 9023 字节进行测试,统计在设定的测试流量下的每 10000 条流总共所消耗的时间。

2) 设置表的大小为 4023 字节进行测试,统计在设定的测试流量下的每 10000 条流总共所消耗的时间。

3) 设置表的大小为 1023 字节进行测试,统计在设定的测试流量下的每 10000 条流总共所消耗的时间。

4) 对消耗的时间情况进行分析。

选择不同的哈希表的大小原因,在于不同的大小,将会产生不同程度的冲突。表的大小越小,其产生冲突的概率将越大,相应链表的长度将会越长。此时,程序改进的效果将会表现的更加的明显。选择测试 10000 条流总共消耗的时间的原因在于因为针对一条流的查找耗费的时间将不会表现的那么明显区分,故采用统计 10000 条流和的形式,进行时间统计,当然为了区分更加的明显,可以采用更多的流的形式,这个并不影响。

结果与分析:

1. 表的大小设置为 9023 字节进行测试对比实验

因为采集的数据量非常的庞大, 所以为了方便说明, 只是截取了中间的 20 组数据, 结果显示如下表 4.4 所示, 数据展示的对比情况如下图 4.11 所示。

表 4.4 表的大小为 9023 字节时各算法使用的时间对比 单位: 微秒

序号	NORMAL 算法	MTF 算法	CACHE 算法
1	63336	58579	59708
2	111809	88761	84349
3	107126	86191	81666
4	117556	90558	86337
5	129161	94770	90314
6	122282	95455	88784
7	158158	111763	103172
8	149454	111367	100830
9	107195	90467	82628
10	108562	90947	82276
11	136878	104223	93697
12	125053	100983	91582
13	117528	97265	88120
14	146911	108626	98724
15	135393	107270	97347
16	120445	100119	91803
17	131659	104003	95317
18	127087	104042	94462
19	124369	103455	94648
20	120723	104250	94922

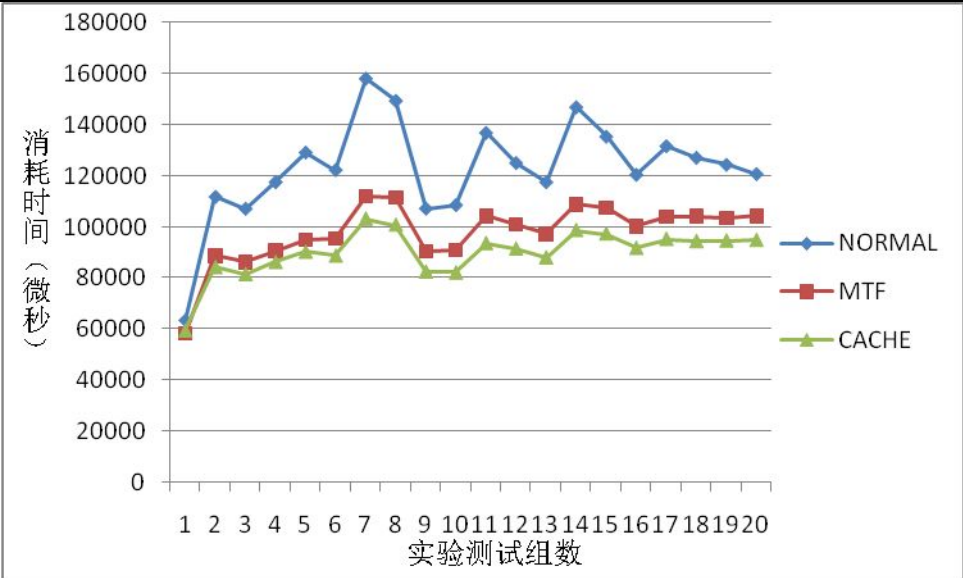


图 4.11 表的大小为 9023 字节的实验对比图

2. 表的大小设置为 4023 字节进行测试对比实验

截取了中间的 20 组数据，结果显示如下表 4.5 所示，数据展示的对比较情况如下图 4.12 所示。

表 4.5 表的大小为 4023 字节时各算法使用的时间对比 单位：微秒

序号	NORMAL 算法	MTF 算法	CACHE 算法
1	66605	60842	59823
2	113496	84979	78988
3	128547	96472	86599
4	132226	96115	87973
5	149157	101741	92617
6	140529	109956	93880
7	146075	110171	94132
8	189527	125149	109535
9	204763	131991	114151
10	193595	133490	114655
11	205539	138652	116904
12	161364	122713	103226
13	153529	117229	99644
14	199751	134505	114065
15	178710	131823	109916

续表 4.5

16	156277	122036	103131
17	173666	127582	108028
18	161892	127649	106851
19	159076	126568	106512
20	151969	127179	106255

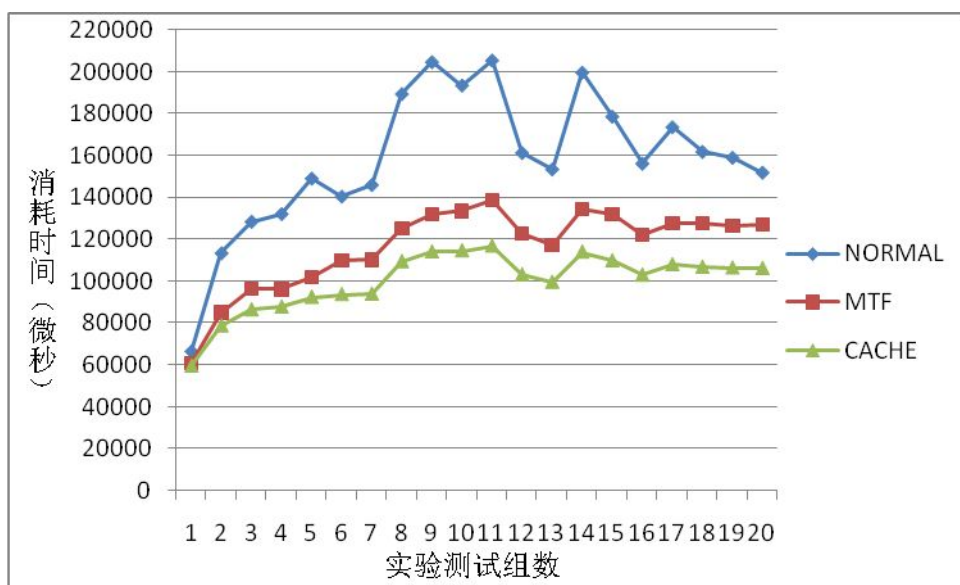


图 4.12 表的大小为 4023 字节的实验对比图

3. 表的大小设置为 1023 字节进行测试对比实验

截取了中间的 20 组数据，结果显示如下表 4.6 所示，数据展示的对比情况如下图 4.13所示。

表 4.6 表的大小为 1023 字节时各算法使用的时间对比 单位：微秒

序号	NORMAL 算法	MTF 算法	CACHE 算法
1	110532	74139	73379
2	404888	208103	166001
3	416844	205525	164274
4	499301	216811	175094
5	447220	210333	167871
6	466896	277893	202169
7	493620	281321	205399
8	730730	332548	259697

续表 4.6

9	786966	346898	265767
10	730022	366950	274578
11	851668	388700	290483
12	775267	379231	278621
13	545653	311416	218138
14	496191	284724	201044
15	746039	357288	260549
16	626703	342733	240698
17	489944	299290	207490
18	582073	319809	226076
19	510270	319555	217446
20	486551	310925	211580

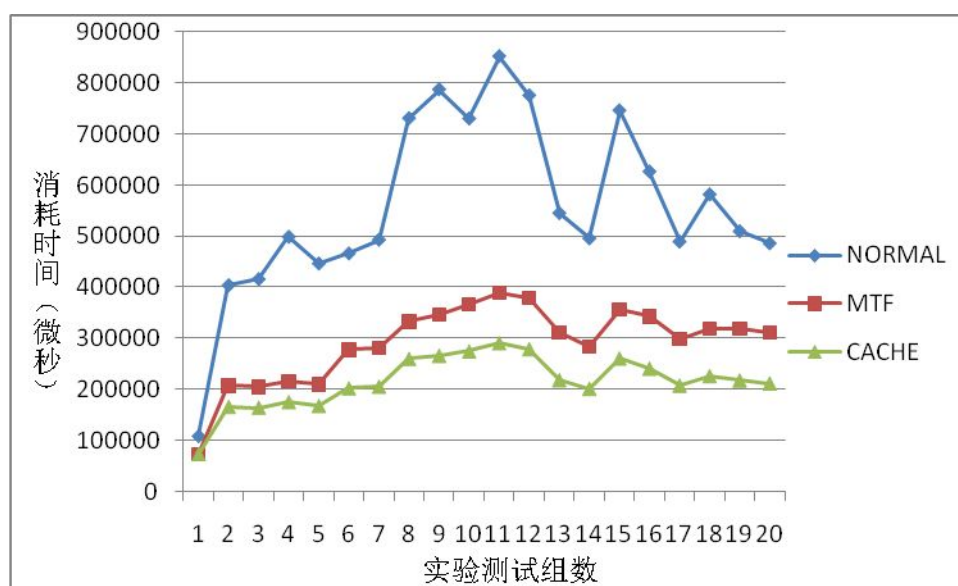


图 4.13 表的大小为 1023 字节的实验对比图

结合上面三组实验数据，得到在不同哈希表大小情况下，MTF 算法消耗时间减去 CACHE 算法消耗时间的差值图，如下图 4.14 所示。

综合实验结果，进行对比分析，由图 4.11、图 4.12、图 4.13 可知，利用局部性的 MTF 算法和 CACHE 算法在效率和性能上相对于最原始的链表 NORMAL 算法，都有较大的提升。而且，可以看到，CACHE 算法的整体效率和性能相对于 MTF 算法，也有相当的改进。就单张图对比，可以看到左侧的时间之差相对较小，随流量递增，时间之差

渐渐增大趋于一种稳定的形式。是因为刚开始的时候，哈希表中数据是空的，产生的冲突比较小，后面随着流量的增加，哈希冲突结点的增加，从而 MTF 算法和 CACHE 算法表现出其优越性。从图 4.14 可以明显的看到，随着哈希表的逐渐减小，哈希冲突的增加，改进的 CACHE 算法，相比 MTF 算法，在性能上表现更加明显的优化。所以，总结概括来说就是，在冲突链不存在的时候，MTF 算法和 CACHE 算法，并不能很好的表现出其优越性。只有当冲突链存在的时候，MTF 算法相对原始的 NORMAL 算法会进行了改进，而 CACHE 算法，则更进一步进行了提高，更好的利用了时间和空间的局部性。

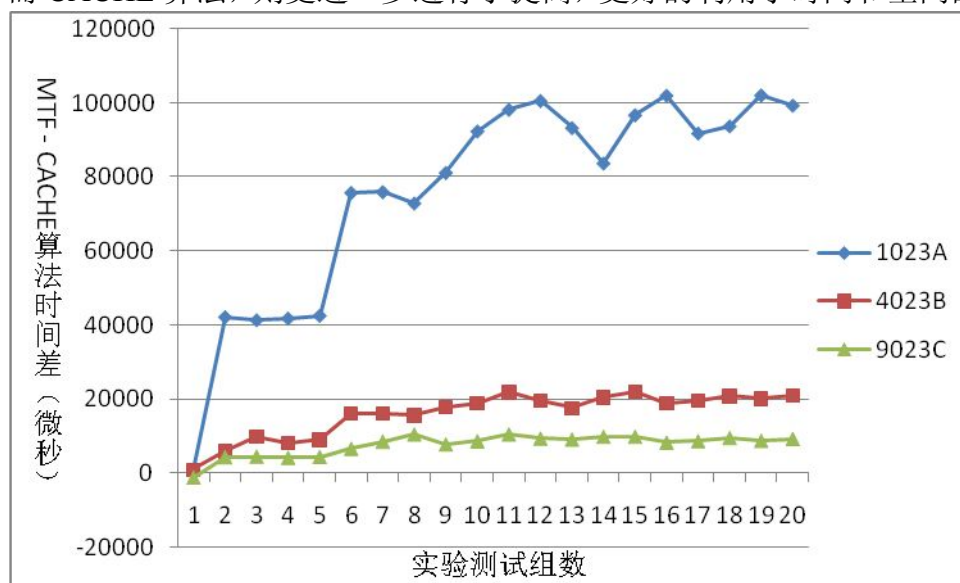


图 4.14 MTF 消耗时间减去 CACHE 消耗时间实验对比图

4.4 本章小结

本章主要讲述的是程序的性能优化，着重介绍了时间、空间的局部性，并且将其引入到程序中进行程序性能的改进。本章主要从两个方面来讲叙关于系统的优化，一个是关于内存池技术，另一个是有关哈希表结点冲突解决技术。并且利用局部性原理，针对原有的算法都进行了优化和改进，在处理性能上得到了进一步的提高，而且通过实验进行验证说明。

结 论

传统的防火墙工作的层次较低，WEB 防火墙工作的层次较高，处理的信息复杂度相比而言会更多，在高层处理的时候，消耗的时间相比传统防火墙而言也会有所增加。随着 WEB 防火墙系统处理流程的复杂化，消耗的时间增多，处在网络出入口上的它容易成为网络通信的性能瓶颈，因此，对它的研究显得越发的重要。

本文对 WEB 防火墙的性能评价进行了钻研，研究了基于排队论理论下防火墙性能评价技术，并且在此基础之上对 WEB 防火墙提出了一个分层排队论模型。该模型将防火墙按照 TCP/IP 模型进行了层次划分，分成了三层排队论系统。三层排队论系统，每一层都可以进行单独的排队论分析，但是三层之间关系紧密，上一层流量的输出是下一层流量的输入，它们之间通过流量进行着关联。本文在分层排队理论的基础之上，主要研究的是系统吞吐量和系统资源分配之间的关系，如何分配系统有限的资源使系统的处理状态达到最优，从而使系统获得最大吞吐量。

此外，本文在性能评价的基础之上，对 WEB 防火墙优化技术进行了钻研，主要从两个方面进行，第一是关于内存的管理技术，第二是关于网络流表及会话表的管理技术。在对这些方面进行研究的时候，发现现有方法的不足并对其改进。

改进的第一个方面是和内存管理相关。有研究者针对链式指针结构管理内存池的缺点，提出了有关以数组下标索引的方式管理动态内存方法，本文在此算法的基础上，修改了索引的存储结构，以及插入弹出的方式，经过实验证明，对其改进，使处理效率得到了提高。

改进的第二个方面是和流及会话管理相关。一般在网络系统中，为了快速定位流和会话，往往采用哈希表来存储相关信息。现有系统往往采用链式解决冲突法，有研究者在其基础之上，融合了网络流的局部性，进行了改进。本文对改进后的方法进行了改进。通过采用空间换时间的方法，在基表中增加多个缓存指针，避免了每次的链表移动。经过实验证明，对其改进，使得总的查询效率和性能得到了提高。

参考文献

- [1] 2013 年中国网民信息安全状况研究报告[EB/OL].http://www.cnnic.net.cn/hlwfzyj/hlwxyzbg/mtbg/201312/t20131219_43475.htm,2013-12-19.
- [2] 国家网络安全宣传周进入电信日 专家：恶意程序呈增长态势[EB/OL].
http://china.cnr.cn/ygxw/201411/t20141128_516912582.shtml,2014-11-28.
- [3] 我国网络安全发展现状及形势展望[EB/OL].<http://finance.qq.com/a/20140311/017792.htm>,2014-03-11.
- [4] 阎慧,王伟,宁宇鹏.防火墙原理与技术[M].机械工业出版社,2004:1-2.
- [5] 田原,云晓春,朱晓晖.防火墙性能基准测试研究[J].计算机仿真,2003,20(7): 123-126.
- [6] Bonnard A, Wolff C. Gesicherte Verbindung von Computernetzwerken mit Hilfe einer Firewall[J]. Studie der SIEMENS AB Zentralabteilung Technik, München, 1997.
- [7] Fulp E W. Parallel firewall designs for high-speed networks[C]//INFOCOM 2006. 25th IEEE International Conference on Computer Communications. Proceedings. IEEE, 2006: 1-4.
- [8] 林闯.计算机网络和计算机系统的性能评价[M].北京:清华大学出版社,2001.
- [9] Salah K. Queuing analysis of network firewalls[C]//Global Telecommunications Conference (GLOBECOM 2010), 2010 IEEE. IEEE, 2010: 1-5.
- [10] Salah K, Elbadawi K, Boutaba R. Performance modeling and analysis of network firewalls[J]. Network and Service Management, IEEE Transactions on, 2012, 9(1): 12-21.
- [11] 侯整风, 庞有祥. 多核防火墙分层内容过滤的时延分析[J]. 计算机工程与应用, 2011, 47(12): 93-96.
- [12] 陈军, 柳国杰. 构建防火墙的性能评价模型[J]. 信阳师范学院学报: 自然科学版, 2006, 18(4): 482-485.
- [13] 徐光辉.随机服务系统(第二版).北京.科学出版社.1988.
- [14] Hoffman D, Prabhakar D, Strooper P. Testing iptables[C]//Proceedings of the 2003 conference of the Centre for Advanced Studies on Collaborative research. IBM Press, 2003: 80-91.
- [15] Yuan L, Chen H, Mai J, et al. Fireman: A toolkit for firewall modeling and analysis[C]//Security and Privacy, 2006 IEEE Symposium on. IEEE, 2006: 15 pp.-213.
- [16] Lyu M R, Lau L K Y. Firewall security: Policies, testing and performance evaluation[C]//Computer Software and Applications Conference, 2000. COMPSAC 2000. The 24th Annual International. IEEE, 2000: 116-121.

- [17]Al-Shaer E S, Hamed H H. Modeling and management of firewall policies[J]. Network and Service Management, IEEE Transactions on, 2004, 1(1): 2-10.
- [18]叶振新, 杨树堂, 马进. 防火墙性能优化[J]. 信息安全与通信保密, 2009 (6): 48-51.
- [19]Al-Shaer E. Dynamic Firewall Configuration Optimization[M]//Automated Firewall Analytics. Springer International Publishing, 2014: 95-127.
- [20]Ganesh A, Sudarsan A, Vasu A K, et al. IMPROVING FIREWALL PERFORMANCE BY USING A CACHE TABLE[J]. network, 2014, 1: 2.
- [21]Zhao Q, Rabbah R, Wong W F. Dynamic memory optimization using pool allocation and prefetching[J]. ACM SIGARCH Computer Architecture News, 2005, 33(5): 27-32.
- [22]Walls K G. Method for improving the performance of dynamic memory allocation by removing small memory fragments from the memory pool: U.S. Patent 5,675,790[P]. 1997-10-7.
- [23]张国兵, 曾武, 黄皓. 用双链哈希表提高基于网络处理器的防火墙性能[J]. 计算机应用, 2006, 25(12): 2742-2744.
- [24]熊兵, 廖年冬, 李峰, 陈晓苏等. 基于网络数据流局部性的连接管理优化方法[J]. 计算机工程, 2011, 37(24): 86-87.
- [25]周伟. 高性能 HTTP 代理服务器关键技术研究实现[D]. 哈尔滨工程大学, 2012.
- [26]Williamson C. Internet traffic measurement[J]. Internet Computing, IEEE, 2001, 5(6): 70-74.
- [27]Gulati N, Williamson C L, Bunt R B. Local Area Network Traffic Locality: Characterization and Application[M]//Local area network interconnection. Springer US, 1993: 233-250.
- [28]郑卫斌, 段中兴, 高磊, 张德运等. 基于 IP 流本地性的状态检测性能优化方法[J]. 西安交通大学学报, 2004, 38(4): 413-416.
- [29]Benchmarking Terminology for Network Interconnection Devices[EB/OL]. <http://www.ietf.org/rfc/rfc1242>.
- [30]Benchmarking Terminology for Network Interconnection Devices[EB/OL]. <http://www.ietf.org/rfc/rfc2647>.
- [31]陆传赉. 排队论 (第 2 版) [M]. 北京: 北京邮电大学出版社, 2009: 1-2.
- [32]唐应辉, 唐小我. 排队论基础与分析技术[M]. 北京: 科学出版社, 2006: 1-11.
- [33]Gunter Bolch, Stefan Greiner, Hermann de Meer, Kishor S. Trivedi. Queueing Networks and Markov Chains Modeling and Performance Evaluation with Computer Science Applications (Second Edition)[M]. New Jersey : John Wiley & Sons, 2006.
- [34]单征. 基于分层架构的网络处理器系统性能分析方法研究[D]. 解放军信息工程大学,

2007.

- [35]Andrew S. Tanenbaum.Modern Operating Systems[M] 北 京 :China Machine Press, 2009:182.
- [36]严蔚敏.数据结构(c 语言版)[M].北京:清华大学出版社,2006:193-198,251-258.
- [37]Donald E. Knuth. 计算机程序设计艺术(第 3 卷). 北京:国防工业出版社, 2002: 485-486.
- [38]Randal E. Bryant, David R. O'Hallaron. Computer systems : a programmer's perspective [M]北京:China Machine Press,2011:406-413.
- [39]R. Caceres, P. Danzig, S. Jamin and D. Mitzel, "Characteristics of Wide-Area TCP/IP Conversations",Proceedings of ACM SIGCOMM, Zürich, Switzerland, pp. 101-112, September 1991.
- [40]Crovella M E, Bestavros A. Self-similarity in World Wide Web traffic: evidence and possible causes[J]. Networking, IEEE/ACM Transactions on, 1997, 5(6): 835-846.
- [41]Arlitt M F, Williamson C L. Internet web servers: Workload characterization and performance implications[J]. IEEE/ACM Transactions on Networking (ToN), 1997, 5(5): 631-645.
- [42]V. Paxson and S. Floyd, "Wide-Area Traffic: The Failure of Poisson Modeling", IEEE/ACM Transactionson Networking, Vol. 3, No. 3, pp. 226-244, June 1995.

致 谢

硕士三载，工程七年。倏忽之间，悄然而逝。舍与不舍，终将过去。留与不留，终将离去。情绕于心，岂易言表。情之为物，易喜易悲。或因一言而泪流满面，或因一言而大笑不已，皆为常事。然终究留于记忆深处，或某一日，因合其物、其景，或有所感，不得而知。

承蒙杨武老师的教诲和关心，感恩在心。也许没有他，当时我也来不了这个实验室。谢玄世昌老师悉心教导，常以做人之道融于技术之中，让我有醍醐灌顶之感。在京岁月，偶尔相伴，谆谆教诲，不曾忘记。论文之道，言谈身教，如无启发，不曾完成。谢王巍老师的三年关怀和照顾，诲我不倦。谢苟大鹏老师一起醉酒言谈的朋友之间的感情，让我感动于心。

还记得四年前，刚保研进入实验室那会，何尝不懵懂无知。写了一手乱七八糟的代码，还沾沾自喜，自以为还可以。然只不过是一把破扫帚，自己尚自珍惜罢了。谢刚入门时，赵彦柏，人称赵公子师兄的指导，让我知道自己，原来这么的不足，也让我们见识了一下大牛的风范。谢张萌师姐，一个为人很好的师姐，耐心的给我们讲着关于我们入门项目的知识，让我们更好的入门。谢小王巍师兄和明瑞阳师兄，在我刚刚进入项目组时，对项目经验的传授。或许在师兄师姐当中，最要感谢的是刘彦龙师兄，我们项目组组长。其实组长只是次要而已。对于项目中许多不明白的地方，给以详加解释和指导，一步一步的带着我成长。让我记忆深刻的地方，是一个周末，大家去聚餐吃饭了，实验室就两个人，我和师兄，我在做作业，遇到不明白的地方，他以前做过，因为时间久了忘了，便帮我一起查资料。在实验室的日子，颇多受到师兄师姐的照顾，深深表示感谢，或许因为受他们的影响，所以我对后面的师弟师妹们也很好的缘故吧。

或许对我影响更多的在于我们这届吧。感谢刘坚。对我的性格有颇多的影响。如果说性格影响分四份，他应该占了一份。他外在张扬的性格，活泼高涨的激情，都影响着我。并不是每一个人在玩的时候总会叫上你，他是其中一个。并不是每一个人，都愿意与你分享成长的痛苦和快乐，他是其中一个。并不是每一个人都有耐心聊一些关于人生的话题，他还是其中的一个。对于此，表示深深的感谢。感谢朱秋阳，她也占了一份，乐观而开朗的性格深深影响着我。一起上课的日子不会被忘记。一起玩闹的日子不会被忘记。一起成长的日子不会被忘记。再次，要感谢的是岳国栋。还记得当年从北京回学校考六级的那些天，天天叫上我，和我一起吃饭的人是他，这些微小的举动不会被忘记。感谢唐芳伟，永远是那么的温和而有耐心值得我去学习，或许因为性格的颇多相似，所以相处的没有太多的压力，一起游戏、北京一起早起的日子已经留在了记忆之中，是我乐意找他帮我的人之一。感谢安凯歌，对技术的热情永远是那么高涨，认认真真值得学习，这股劲头影响着我而让我羡慕，还有就是每欲其一道，便欣然往之而令人感动。感

谢李翠娇，颇有见解的一个女生。一起为项目奋斗，一起对问题的愤愤不满或许都已经随风而去了，剩下的估计也只有相处快乐的记忆。感谢杨润，原以为是一个玩世不恭的人呢，后来相识日久才知道他做事情是那么的认真。感谢曹亚男、陈泳杉、段茂涛、蔡春龙、孟卓，十三个人一起把酒言欢，大谈欢唱的日子，有你们相伴，真的感谢。

说到性格影响还有两个人还需要感谢那就是刘超和李伟。刘超像大哥一下照顾着我，而李伟以他强大的自信心影响着我。此外，要感谢一下唐匀龙，一起生活七年的好室友，对我的包容和谅解。感谢北京一起生活的朱于增、常飞等师弟，给北京生活带来了太多的欢乐。

父母在，不远游，游必有方。感谢父母的一贯的支持和爱护。还要感谢一下那些关心我的人，无论是直接还是默默，谢谢对我的关心。

最后，对评阅本文的老师们的致以诚挚的谢意！